Convolutional Neural Networks

Jefersson A. dos Santos
j.santos@sheffield.ac.uk

Learning Outcomes

By the end of this lecture, you should be able to:

- Understand basic concepts and principles behind CNNs
- Recognize the main differences between CNNs and Fully-Connected Networks
- Understand the workflow involved in training and evaluating a CNN model for image classification
- Learn strategies for optimizing CNN performance

Outline

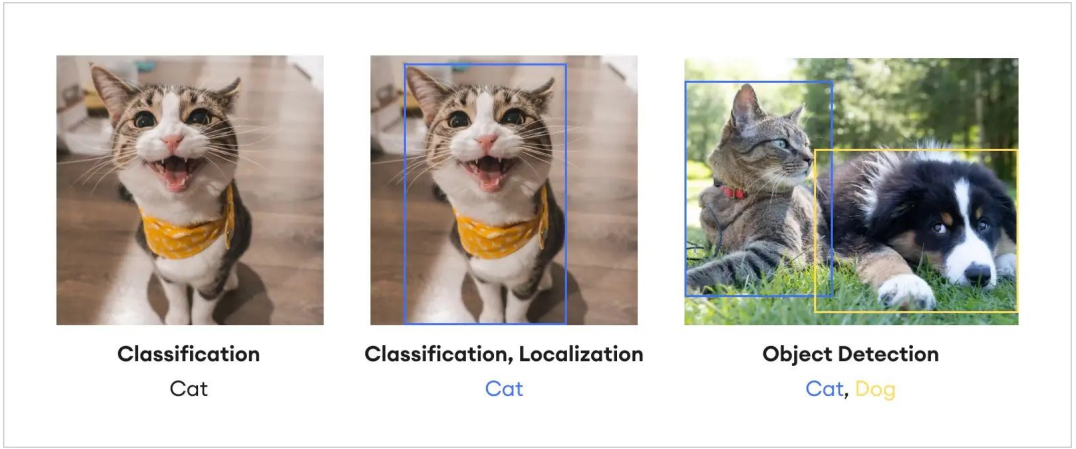
1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - Stride and Padding
    - Pooling
    - Loss functions
3. Convolutional Neural Networks
    - Typical Architecture and Training Process
    - Data Augmentation
    - Transfer Learning and Fine Tuning

Outline

1. **Introduction**
2. Basic Concepts
   - Convolutional Perceptron
   - Convolutional Layer
   - Stride and Padding
   - Pooling
   - Loss functions
3. Convolutional Neural Networks
   - Typical Architecture and Training Process
   - Data Augmentation
   - Transfer Learning and Fine Tuning

Key Questions

Q1 - Is it possible to use MLP for image tasks (e.g. classification?)

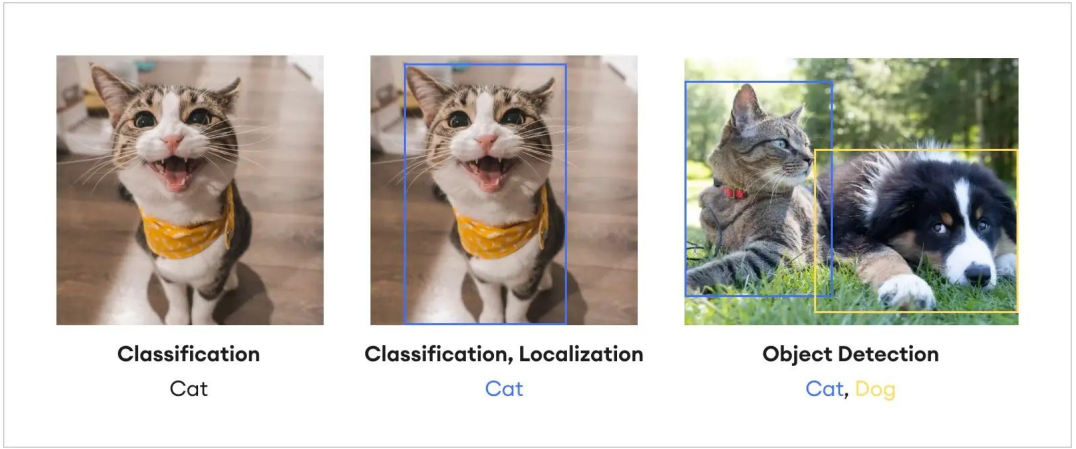


Key Questions

Q1 - Is it possible to use MLP for image tasks (e.g. classification?)

A: yes.

Q2 - Is it worth using MLPs for image tasks?



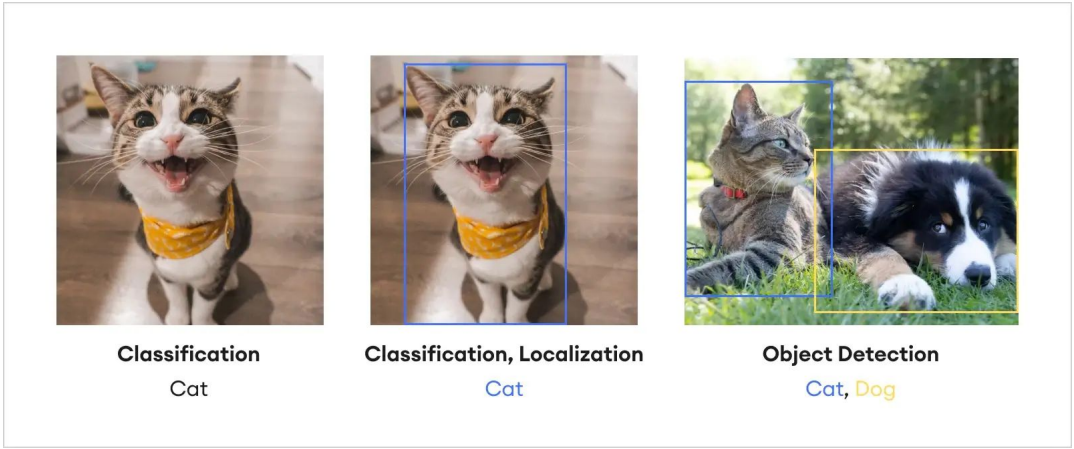
Key Questions

Q1 - Is it possible to use MLP for image tasks (e.g. classification?)

A: yes.

Q2 - Is it worth using MLPs for image tasks?

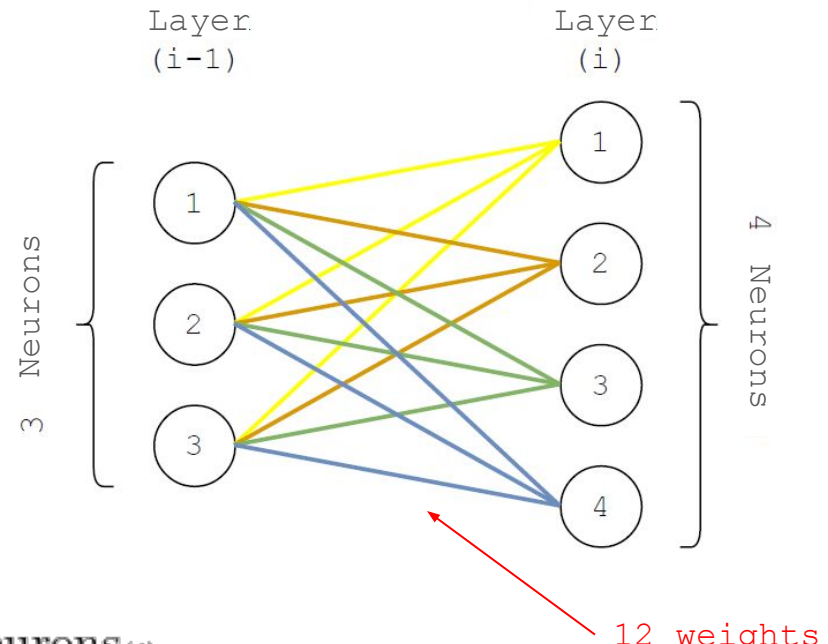
A: No!!



Multilayer Perceptrons

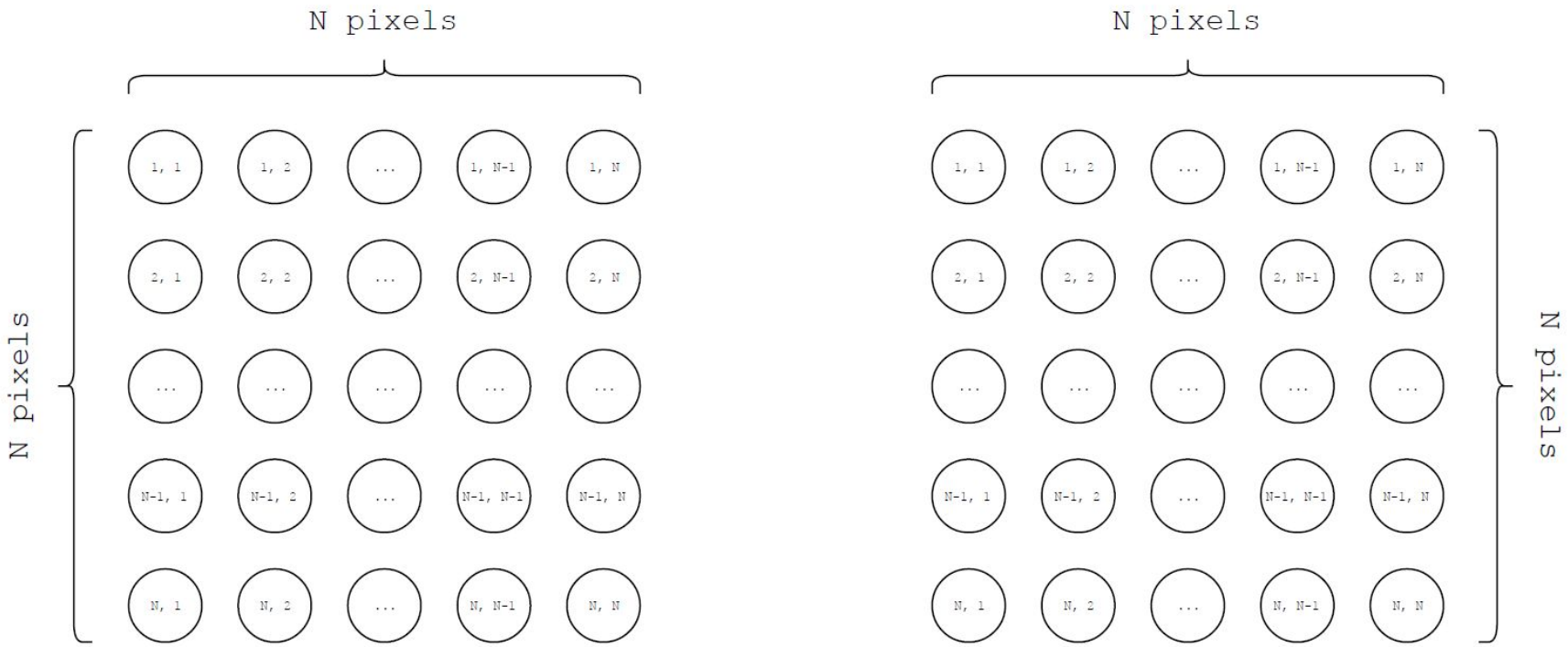
Number of weights:

- #Output neurons in the previous layer
- #Neurons in the current layer

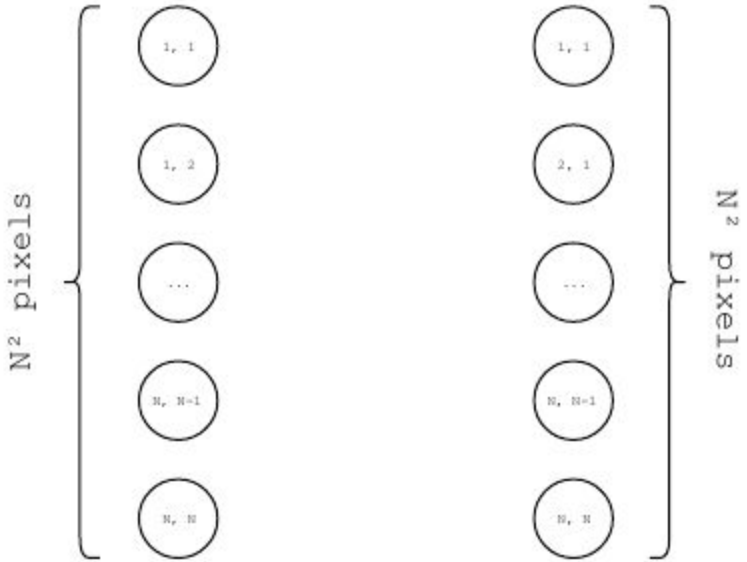


$$\#\text{of weights} = \#\text{of neurons}_{(i-1)} \times \#\text{of neurons}_{(i)}$$

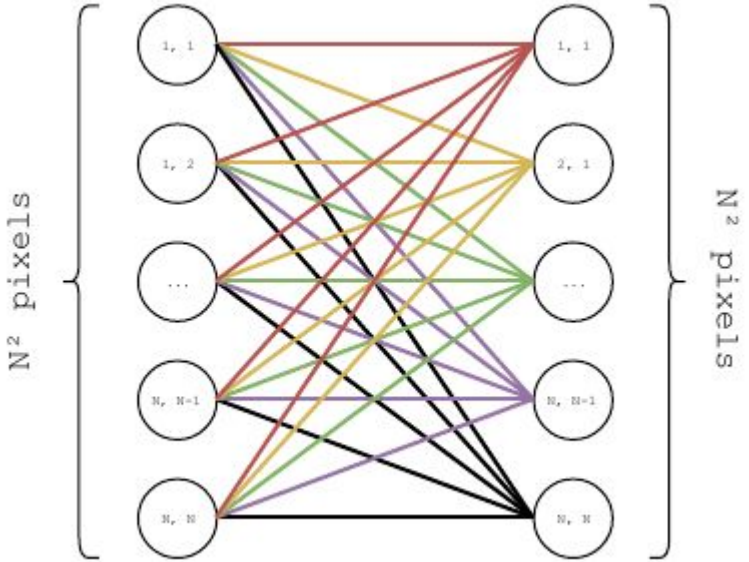
MLP for Images



MLP for Images

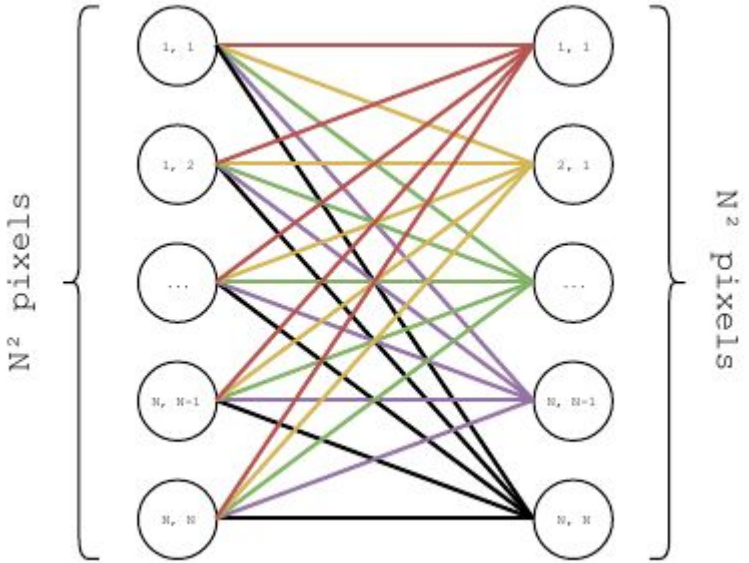


MLP for Images



How many weights?

MLP for Images

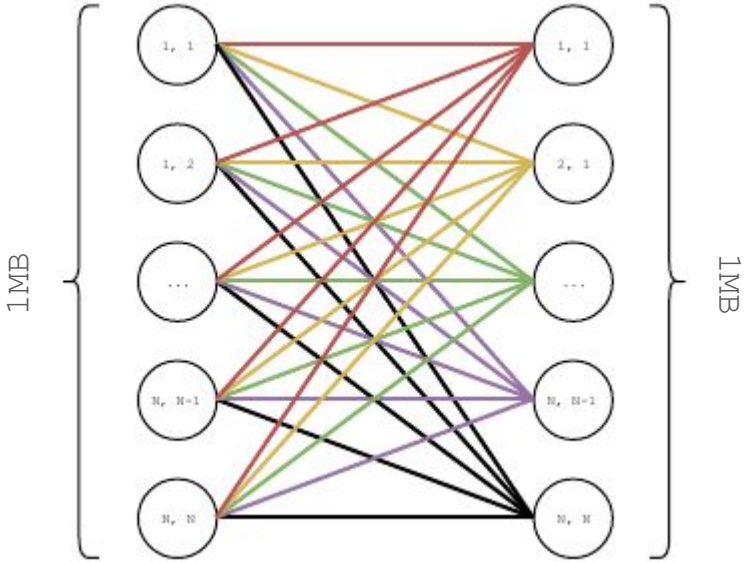


How many weights?

Example: N = 512

# of weights = 512^4 = 68Bi

MLP for Images



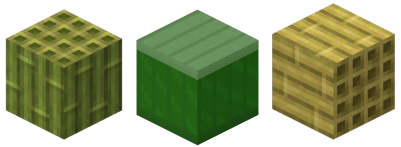
How much memory?

For 1MB images, it will need 256GB

MLP for Images - Limitations

1. Number of neurons in MLPs depends on the input dimensionality of the data.
2. The weights and biases grow polynomially with the number of dimensions of the input image
    - Not suitable to process reasonably large images (i.e. 1024 × 1024).
3. Fully connected characteristics of MLPs
    - impossible to be invariant to the translation of objects in images.
4. Modeling tasks on images using MLPs is limited to a single size of input images.

How to dissociate the number of neurons from the spatial resolution of the input?

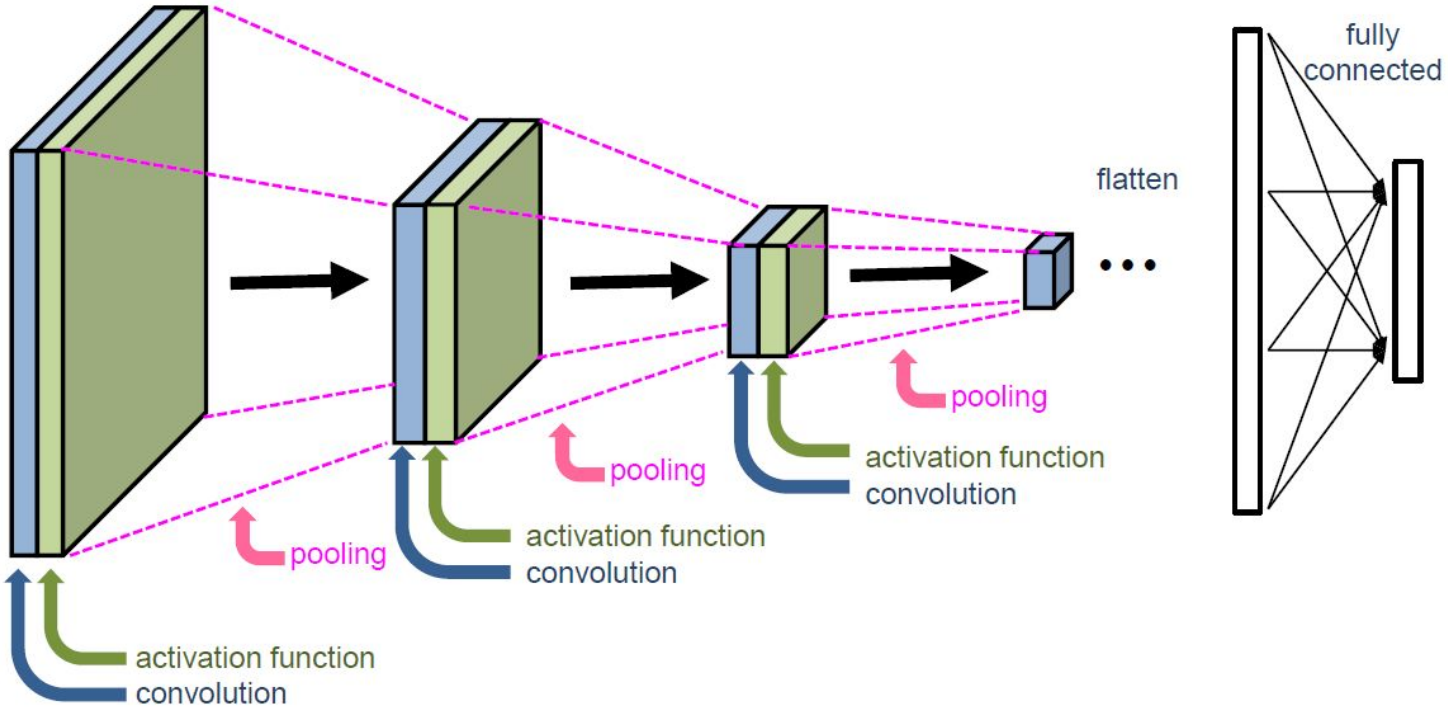


Convolutional Neural Networks

Basic Concepts

Convolutional Neural Networks

Typical Architecture



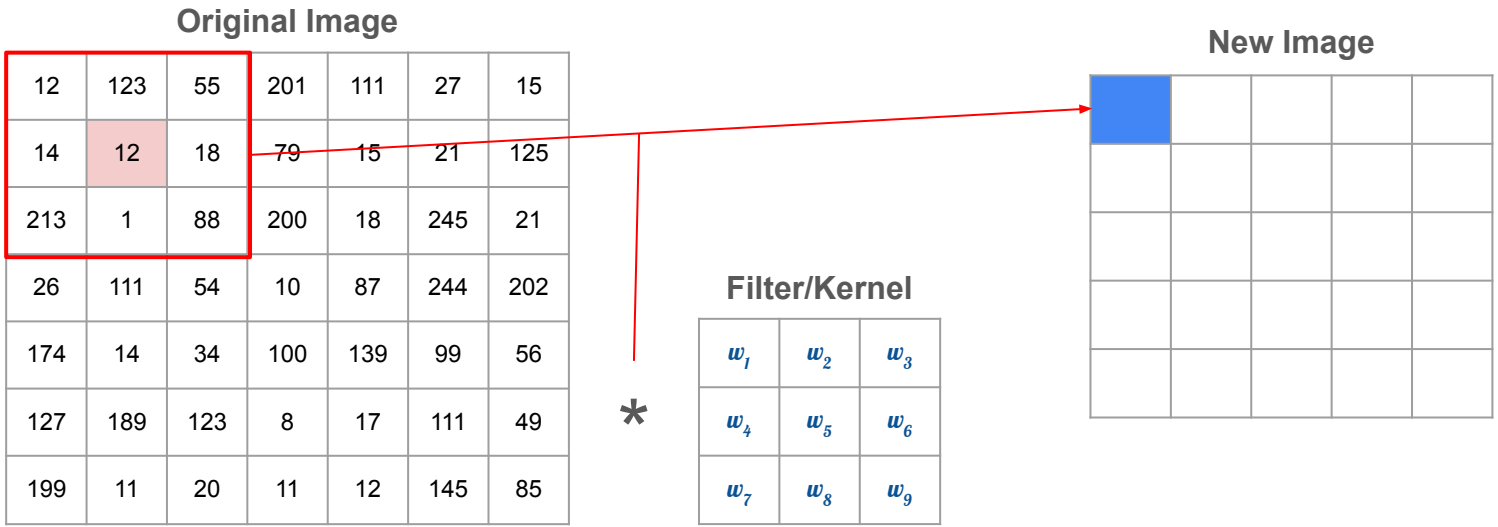
Adapted from Raul Feitosa (PUC-Rio)

Convolution

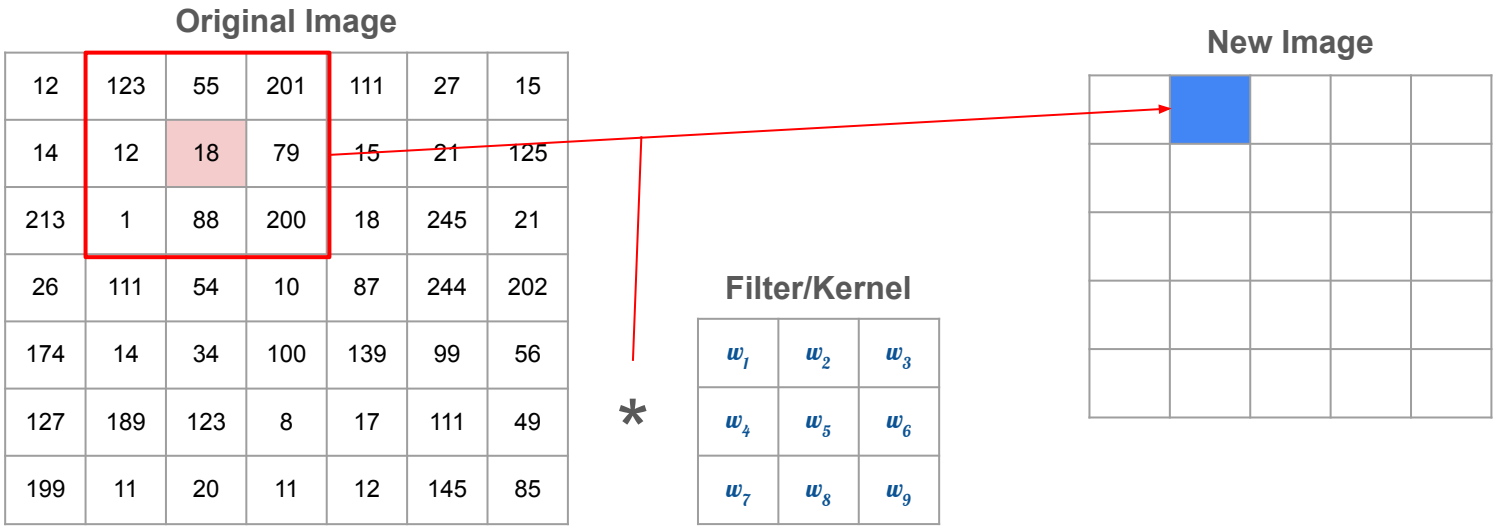
Original Image

12	123	55	201	111	27	15
14	12	18	79	15	21	125
213	1	88	200	18	245	21
26	111	54	10	87	244	202
174	14	34	100	139	99	56
127	189	123	8	17	111	49
199	11	20	11	12	145	85

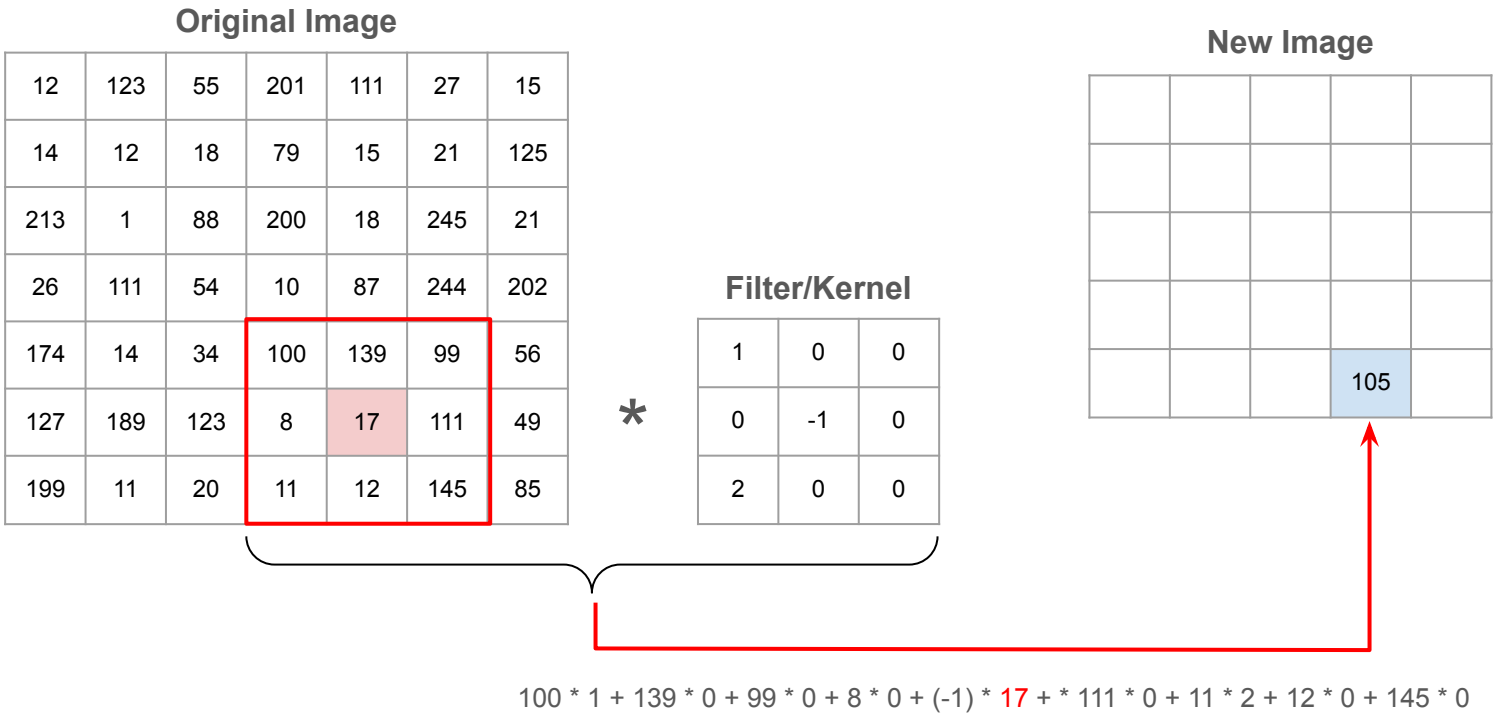
Convolution



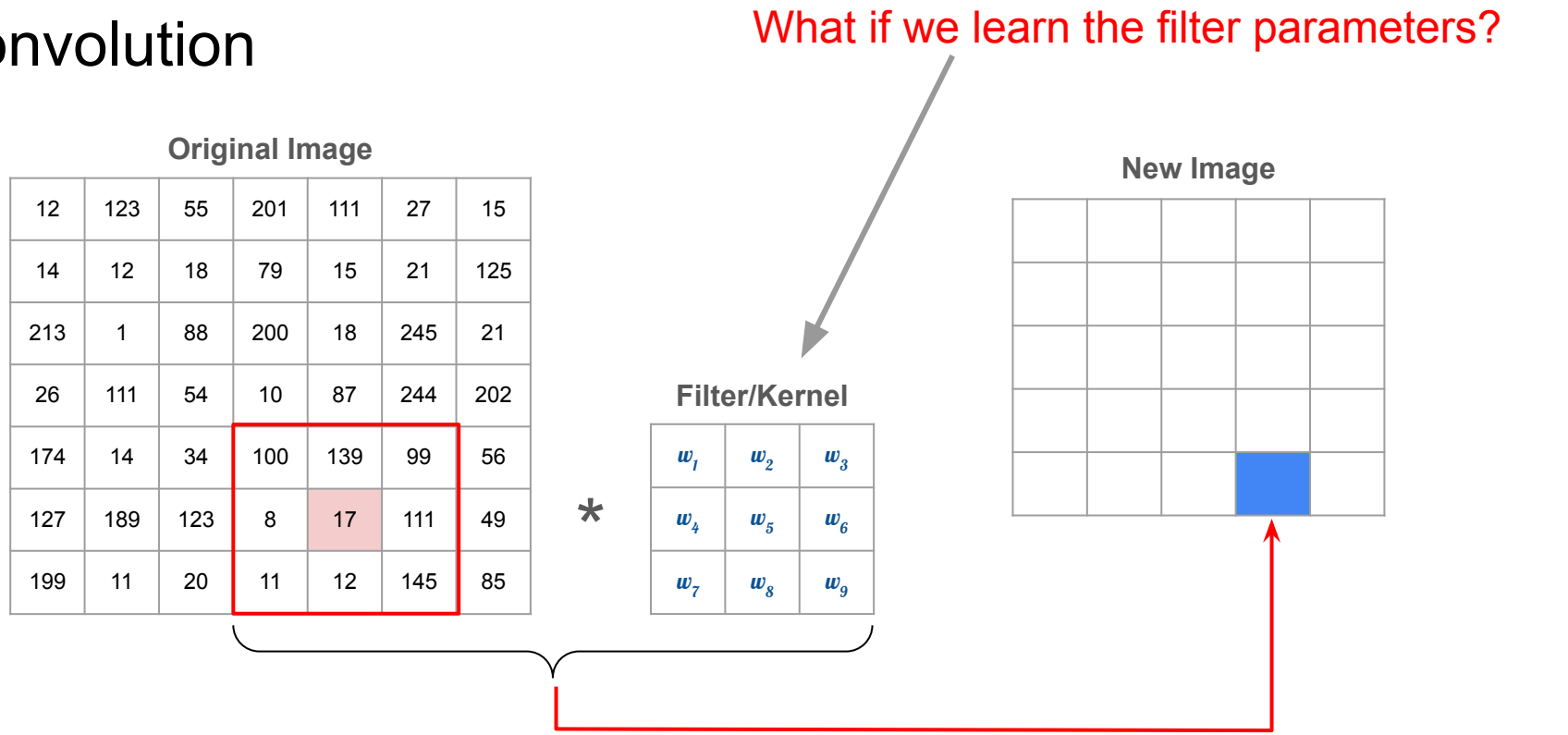
Convolution



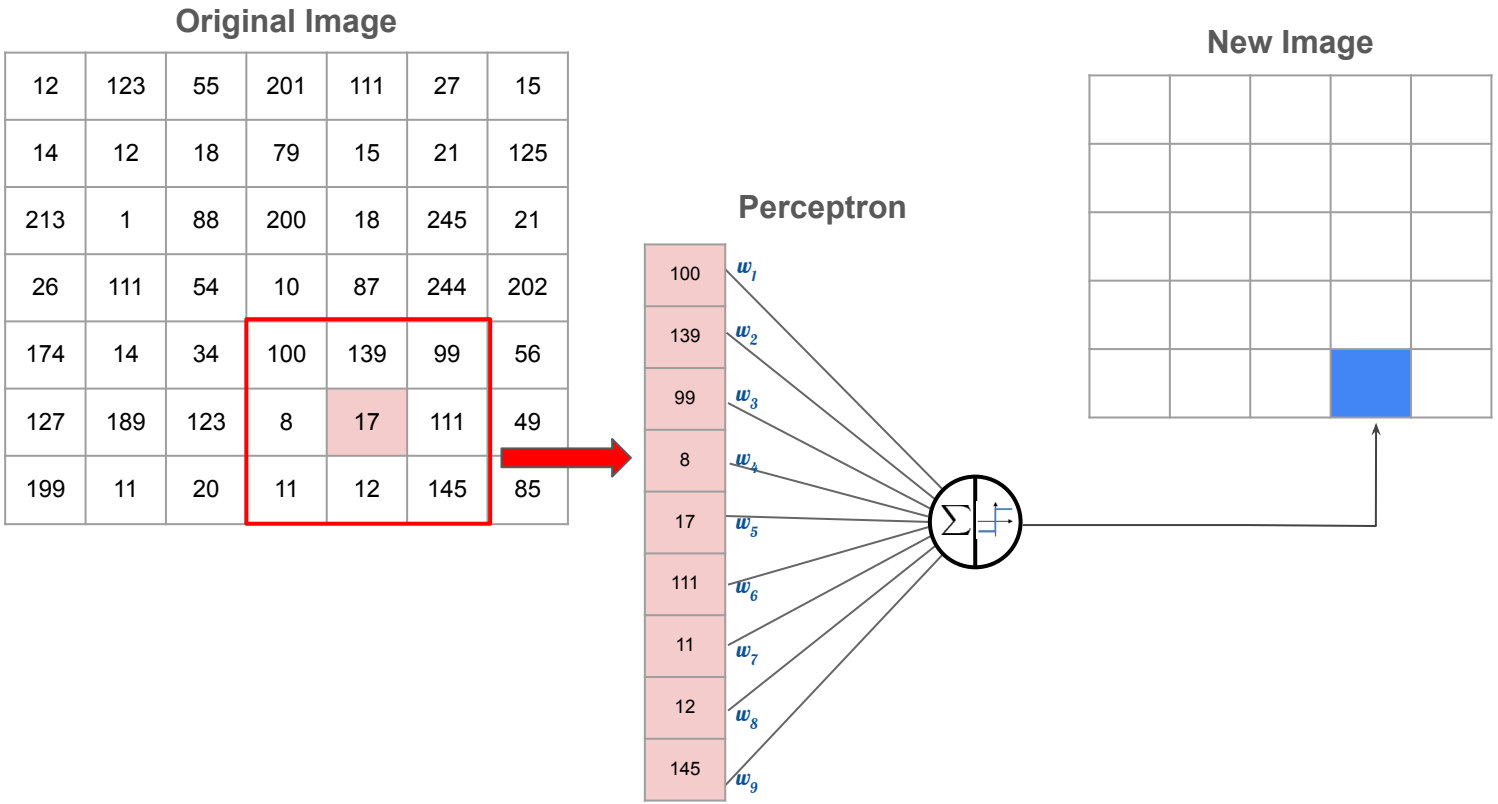
Convolution



Convolution



Convolutional Perceptron



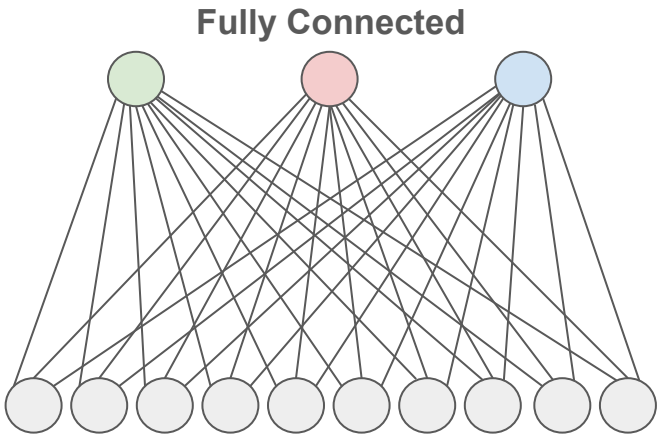
Fully Connected vs Convolutional Units

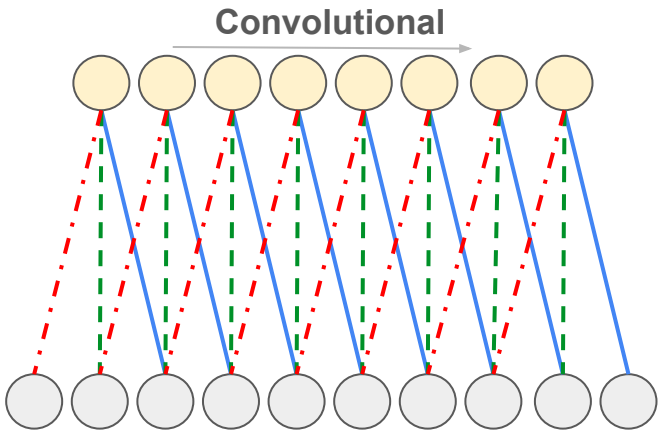
Fully connected layer (left):

- Each unit is connected to all units of the previous layers.

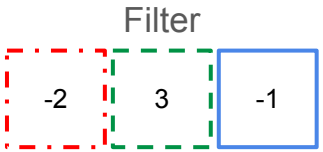
Convolutional layer (right):

- Each unit is connected to a constant number of units in a local region of the previous layer.





Fully Connected vs Convolutional Units

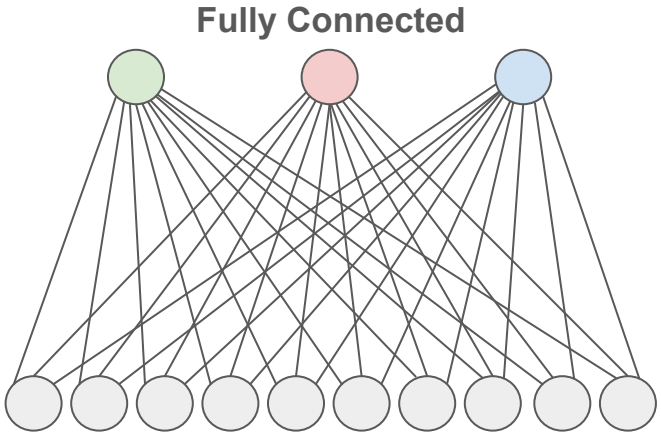


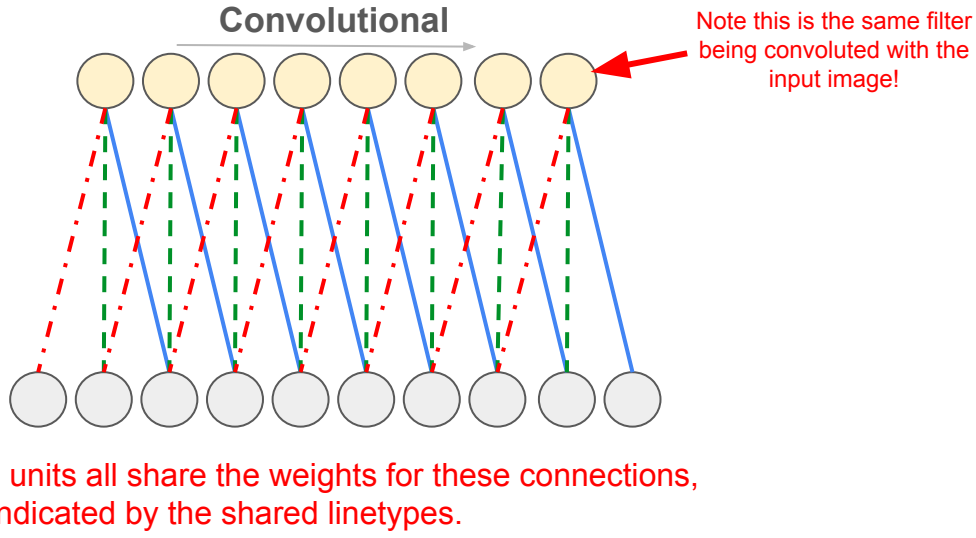
Fully connected layer (left):

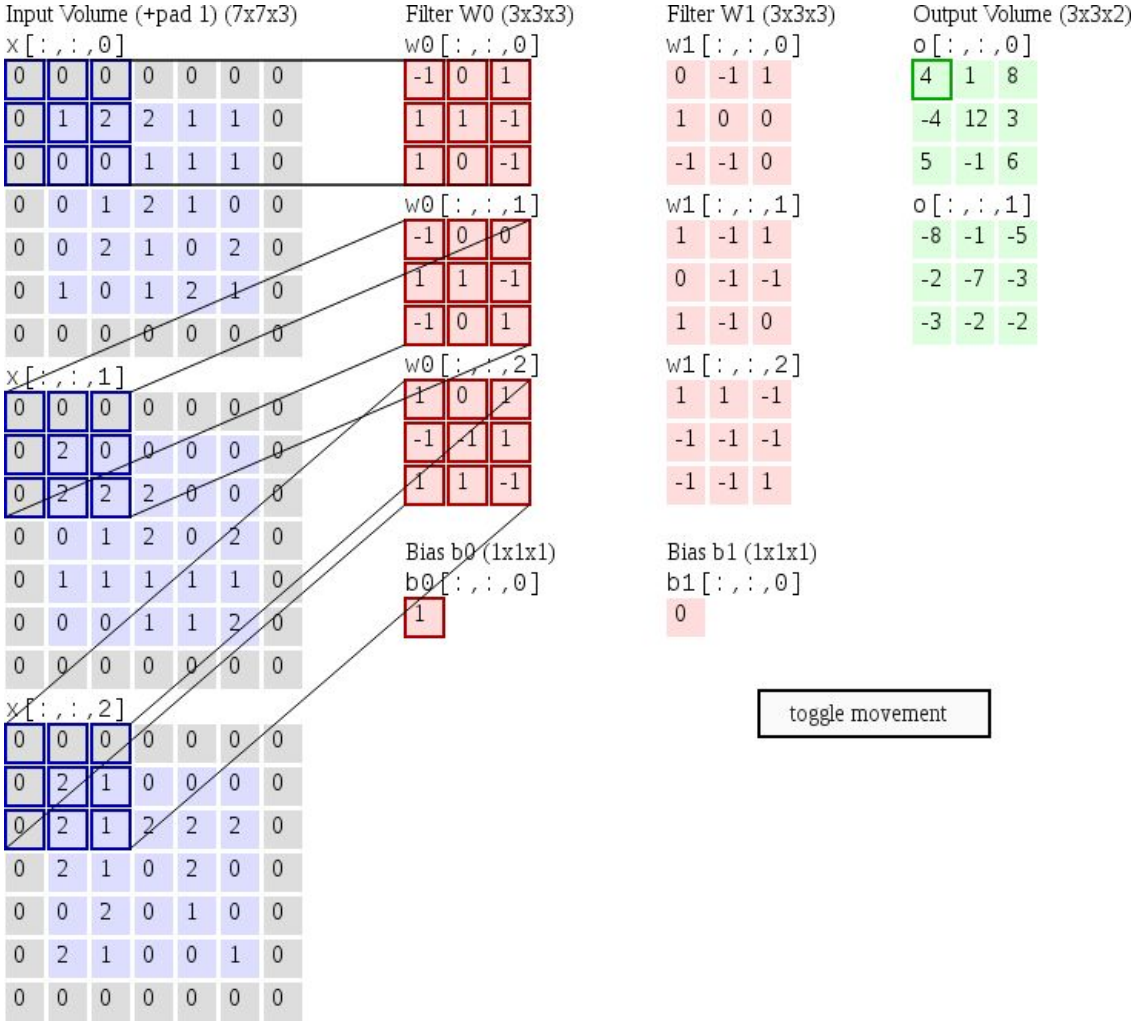
- Each unit is connected to all units of the previous layers.

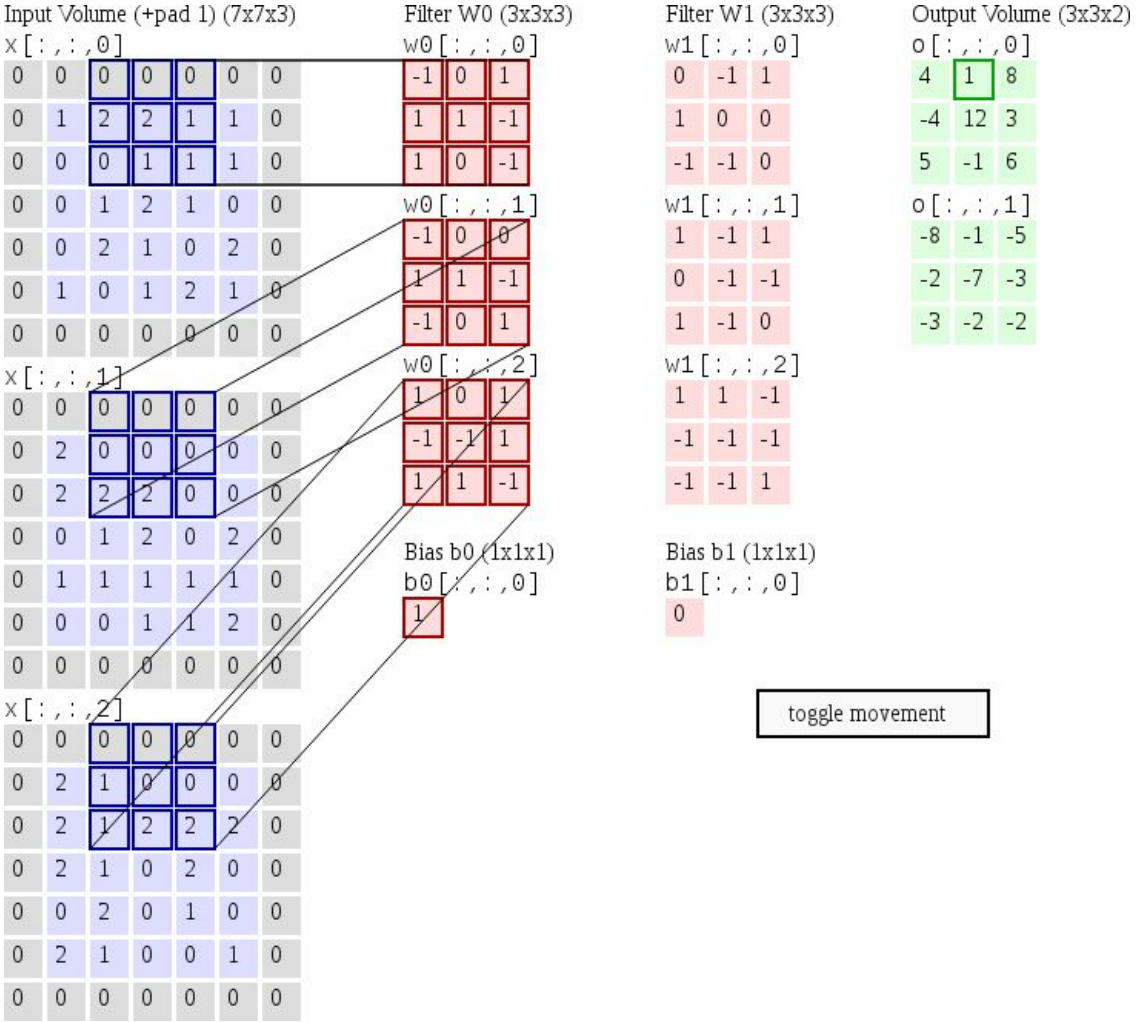
Convolutional layer (right):

- Each unit is connected to a constant number of units in a local region of the previous layer.






Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement

Input Volume (+pad 1) (7x7x3)

x[:,:,0]

0	0	0	0	0	0	0
0	1	2	2	1	1	0
0	0	0	1	1	1	0
0	0	1	2	1	0	0
0	0	2	1	0	2	0
0	1	0	1	2	1	0
0	0	0	0	0	0	0

x[:,:,1]

0	0	0	0	0	0	0
0	2	0	0	0	0	0
0	2	2	2	0	0	0
0	0	1	2	0	2	0
0	1	1	1	1	1	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

x[:,:,2]

0	0	0	0	0	0	0
0	2	1	0	0	0	0
0	2	1	2	2	2	0
0	2	1	0	2	0	0
0	0	2	0	1	0	0
0	2	1	0	0	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:,:,0]

-1	0	1
1	1	-1
1	0	-1

w0[:,:,1]

-1	0	0
1	1	-1
-1	0	1

w0[:,:,2]

1	0	1
-1	-1	1
1	1	-1

Bias b0 (1x1x1)

b0[:,:,0]

1

Filter W1 (3x3x3)

w1[:,:,0]

0	-1	1
1	0	0
-1	-1	0

w1[:,:,1]

1	-1	1
0	-1	-1
1	-1	0

w1[:,:,2]

1	1	-1
-1	-1	-1
-1	-1	1

Bias b1 (1x1x1)

b1[:,:,0]

0

Output Volume (3x3x2)

o[:,:,0]

4	1	8
-4	12	3
5	-1	6

o[:,:,1]

-8	-1	-5
-2	-7	-3
-3	-2	-2

toggle movement

Input Volume (+pad 1) (7x7x3)

x[:,:,0]

0	0	0	0	0	0	0
0	1	2	2	1	1	0
0	0	0	1	1	1	0
0	0	1	2	1	0	0
0	0	2	1	0	2	0
0	1	0	1	2	1	0
0	0	0	0	0	0	0

x[:,:,1]

0	0	0	0	0	0	0
0	2	0	0	0	0	0
0	2	2	2	0	0	0
0	0	1	2	0	2	0
0	1	1	1	1	1	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

x[:,:,2]

0	0	0	0	0	0	0
0	2	1	0	0	0	0
0	2	1	2	2	2	0
0	2	1	0	2	0	0
0	0	2	0	1	0	0
0	2	1	0	0	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:,:,0]

-1	0	1
1	1	-1
1	0	-1

w0[:,:,1]

-1	0	0
1	1	-1
-1	0	1

w0[:,:,2]

1	0	1
-1	-1	1
1	1	-1

Bias b0 (1x1x1)

b0[:,:,0]

1

Filter W1 (3x3x3)

w1[:,:,0]

0	-1	1
1	0	0
-1	-1	0

w1[:,:,1]

1	-1	1
0	-1	-1
1	-1	0

w1[:,:,2]

1	1	-1
-1	-1	-1
-1	-1	1

Bias b1 (1x1x1)

b1[:,:,0]

0

Output Volume (3x3x2)

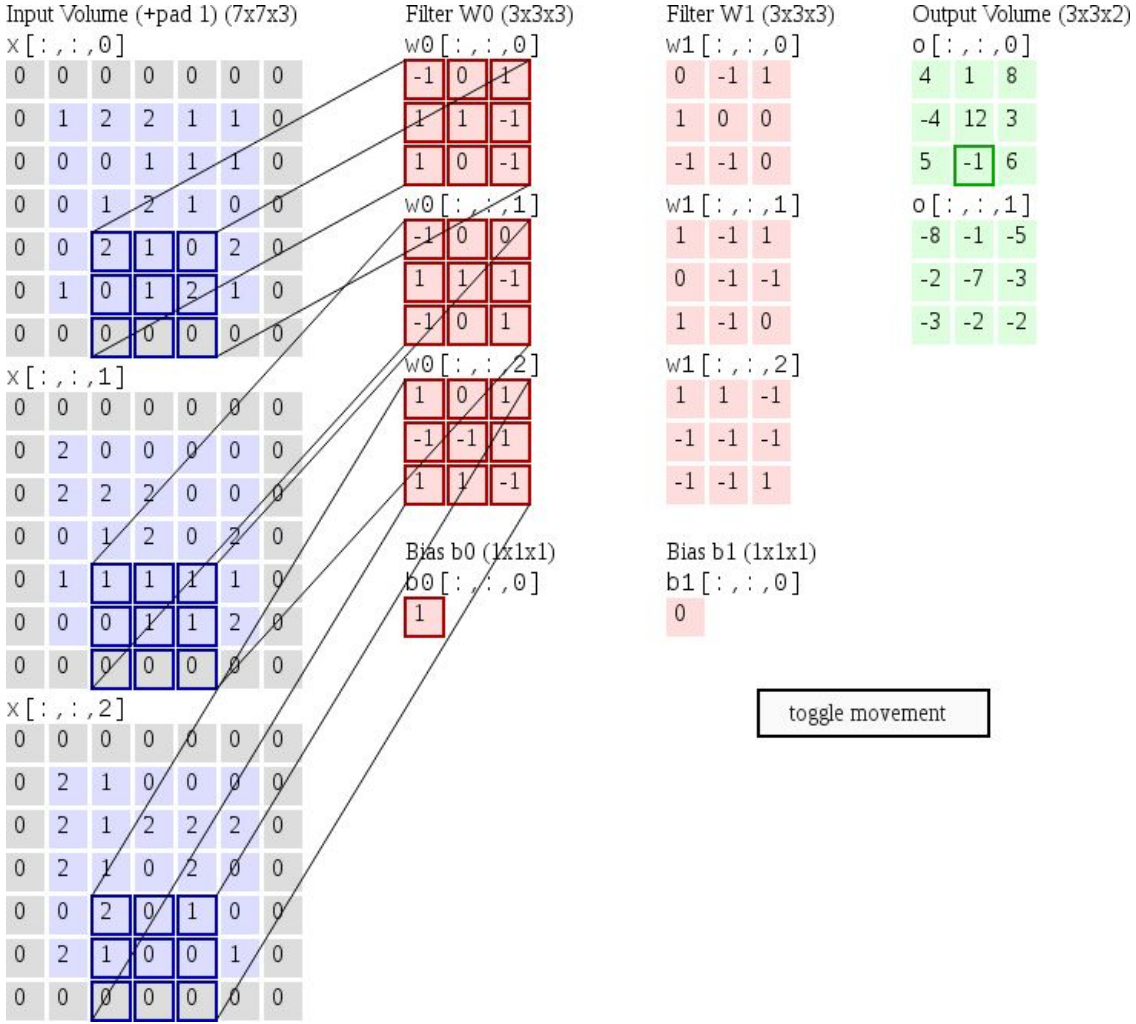
o[:,:,0]

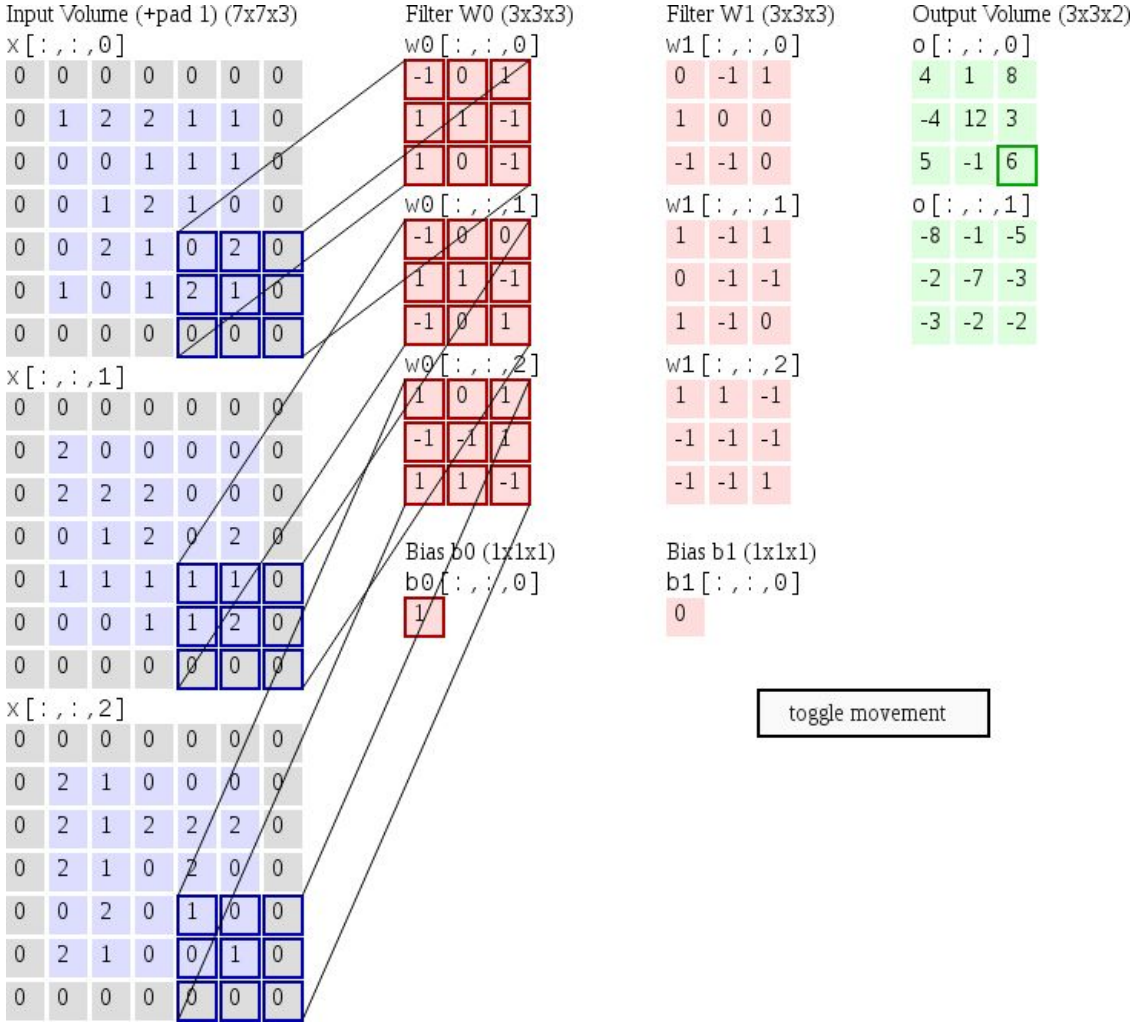
4	1	8
-4	12	3
5	-1	6

o[:,:,1]

-8	-1	-5
-2	-7	-3
-3	-2	-2

toggle movement


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement

Input Volume (+pad 1) (7x7x3)

x[:,:,0]

0	0	0	0	0	0	0
0	1	2	2	1	1	0
0	0	0	1	1	1	0
0	0	1	2	1	0	0
0	0	2	1	0	2	0
0	1	0	1	2	1	0
0	0	0	0	0	0	0

x[:,:,1]

0	0	0	0	0	0	0
0	2	0	0	0	0	0
0	2	2	2	0	0	0
0	0	1	2	0	2	0
0	1	1	1	1	1	0
0	0	0	1	1	2	0
0	0	0	0	0	0	0

x[:,:,2]

0	0	0	0	0	0	0
0	2	1	0	0	0	0
0	2	1	2	2	2	0
0	2	1	0	2	0	0
0	0	2	0	1	0	0
0	2	1	0	0	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:,:,0]

-1	0	1
1	1	-1
1	0	-1

w0[:,:,1]

-1	0	0
1	1	-1
-1	0	1

w0[:,:,2]

1	0	1
-1	-1	1
1	1	-1

Filter W1 (3x3x3)

w1[:,:,0]

0	-1	1
1	0	0
-1	-1	0

w1[:,:,1]

1	1	1
0	-1	-1
1	-1	0

w1[:,:,2]

1	1	-1
-1	-1	-1
-1	-1	1

Output Volume (3x3x2)

o[:,:,0]

4	1	8
-4	12	3
5	-1	6

o[:,:,1]

-8	-1	-5
-2	-7	-3
-3	-2	-2

Bias b0 (1x1x1)

b0[:,:,0]

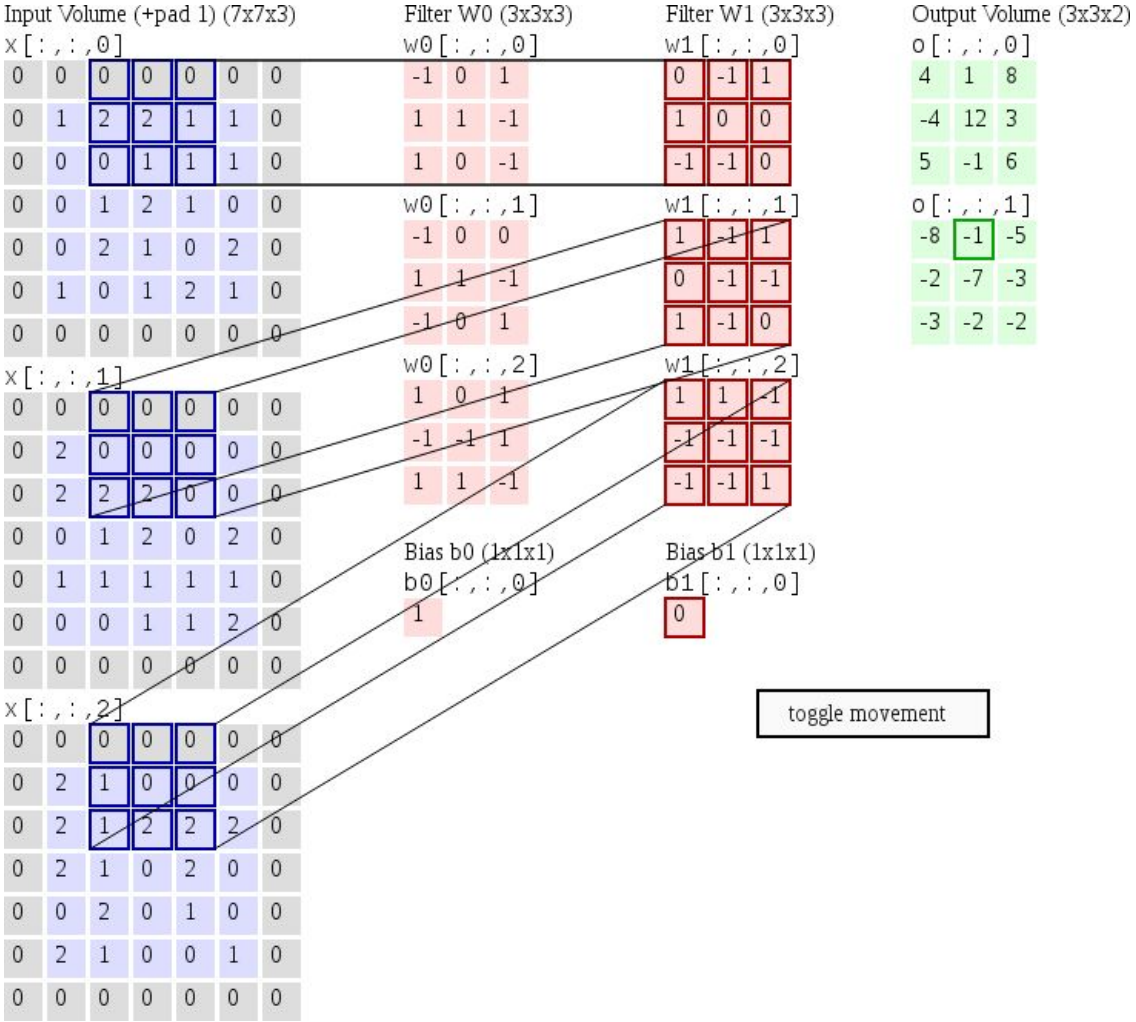
1

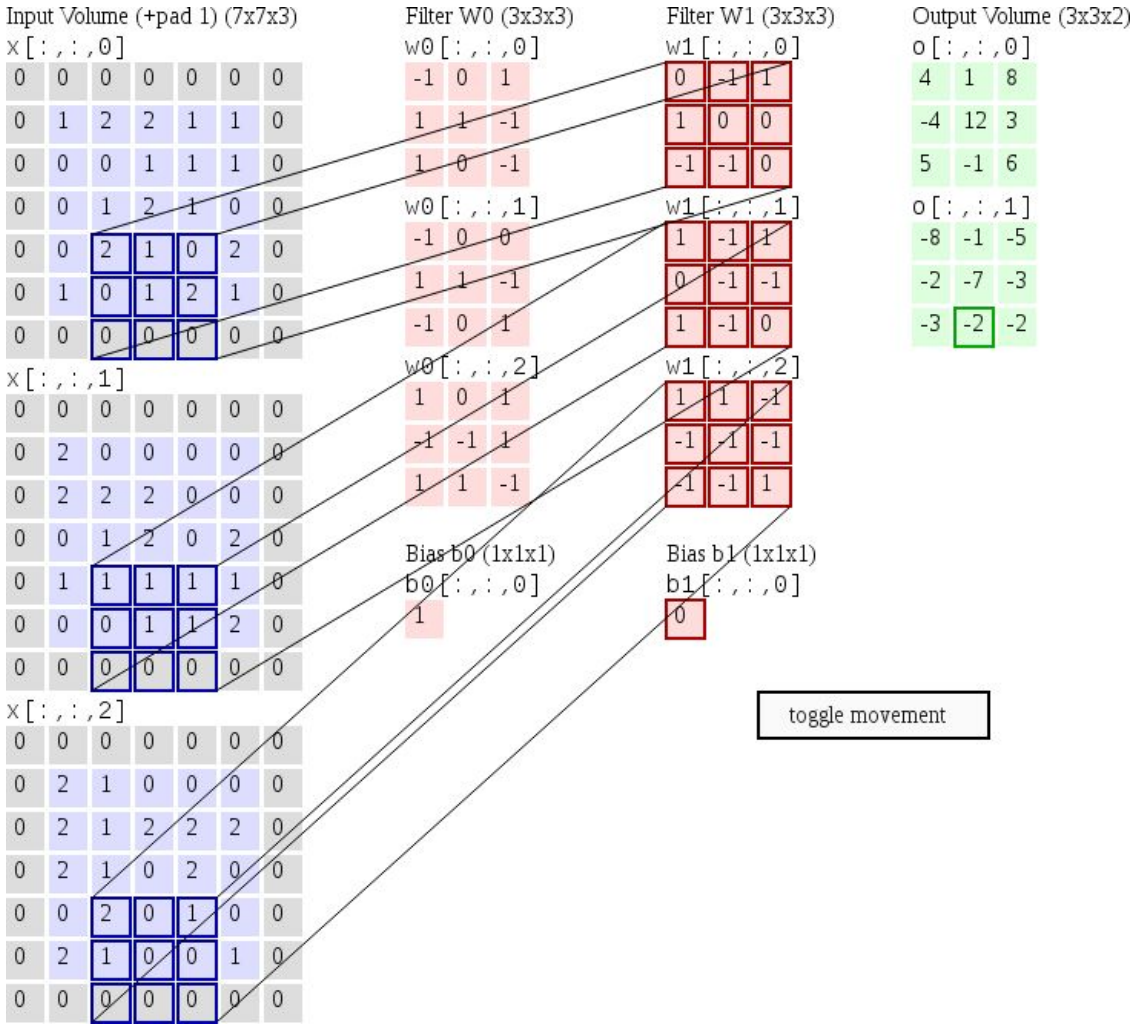
Bias b1 (1x1x1)

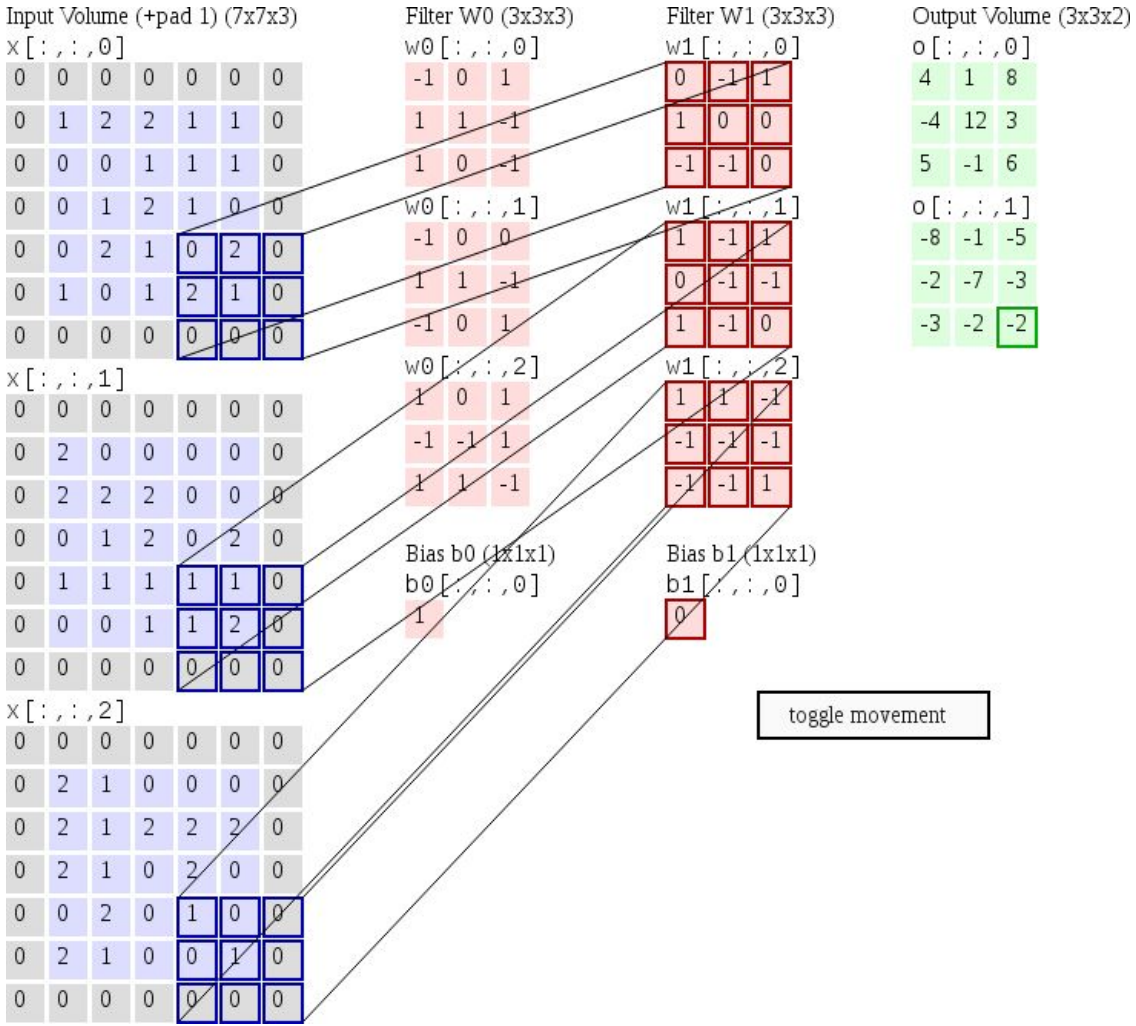
b1[:,:,0]

0

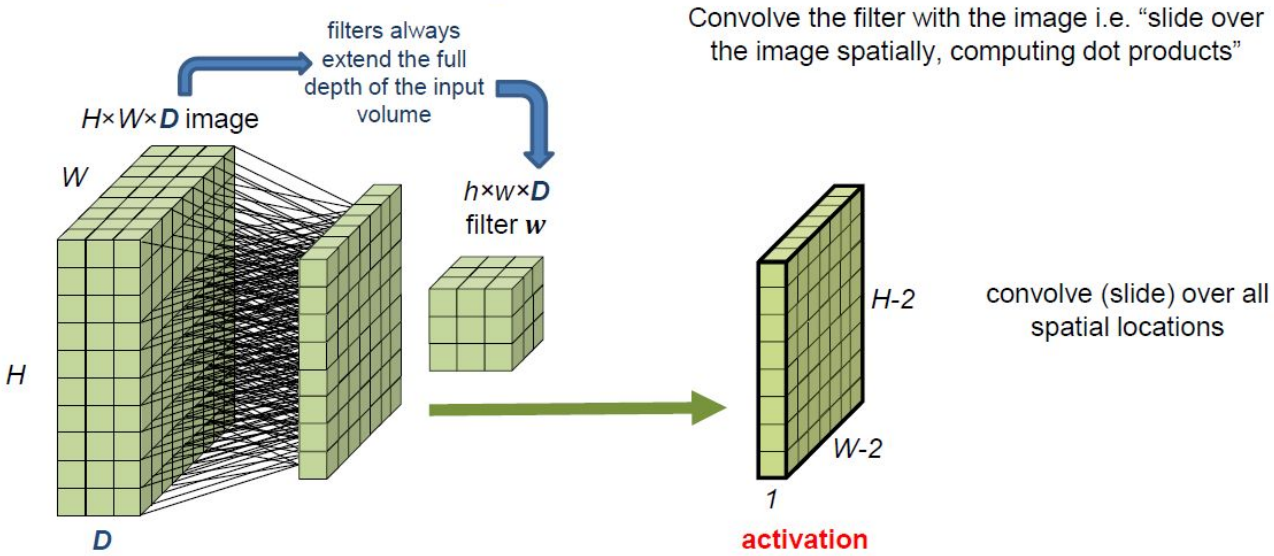
toggle movement


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement

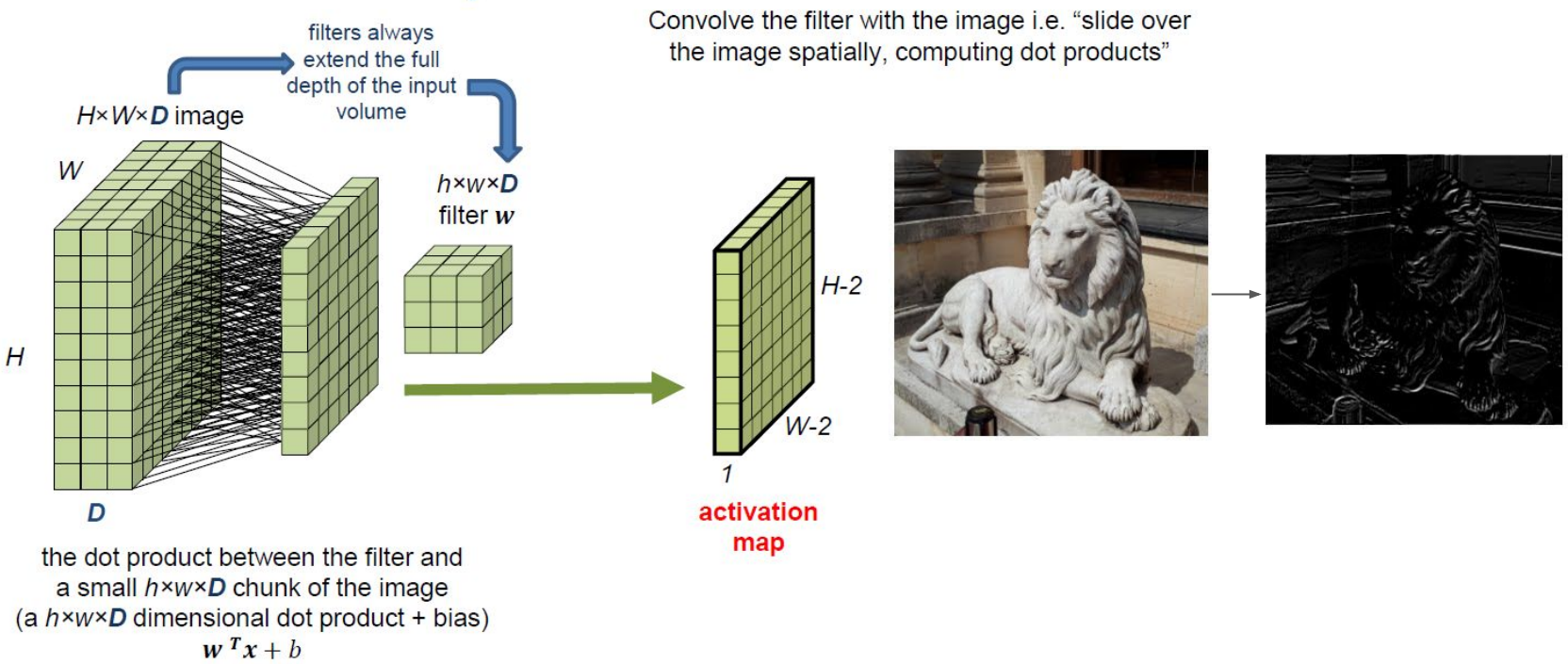

Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 1 2 2 1 1 0
0 0 0 1 1 1 0
0 0 1 2 1 0 0
0 0 2 1 0 2 0
0 1 0 1 2 1 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 0 0 0 0 0
0 2 2 2 0 0 0
0 0 1 2 0 2 0
0 1 1 1 1 1 0
0 0 0 1 1 2 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 0 0 0 0
0 2 1 2 2 2 0
0 2 1 0 2 0 0
0 0 2 0 1 0 0
0 2 1 0 0 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
1 1 -1
1 0 -1
w0[:,:,1]
-1 0 0
1 1 -1
-1 0 1
w0[:,:,2]
1 0 1
-1 -1 1
1 1 -1
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 -1 1
1 0 0
-1 -1 0
w1[:,:,1]
1 -1 1
0 -1 -1
1 -1 0
w1[:,:,2]
1 1 -1
-1 -1 -1
-1 -1 1
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
4 1 8
-4 12 3
5 -1 6
o[:,:,1]
-8 -1 -5
-2 -7 -3
-3 -2 -2
toggle movement

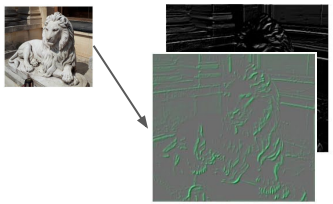
Convolutional Layer



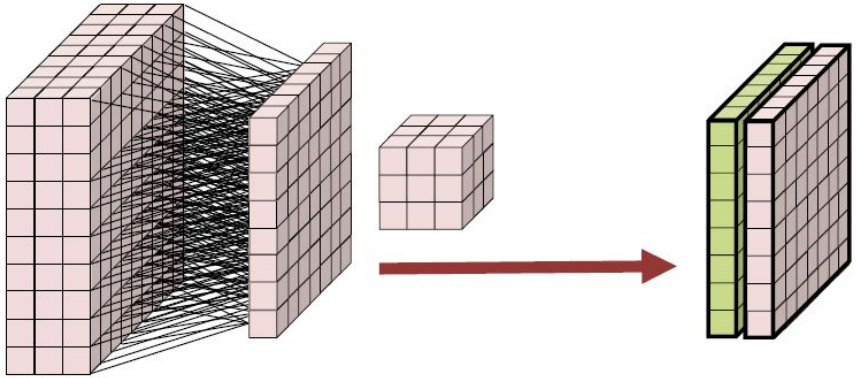
Convolutional Layer



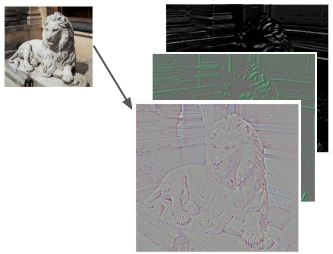
Convolutional Layer



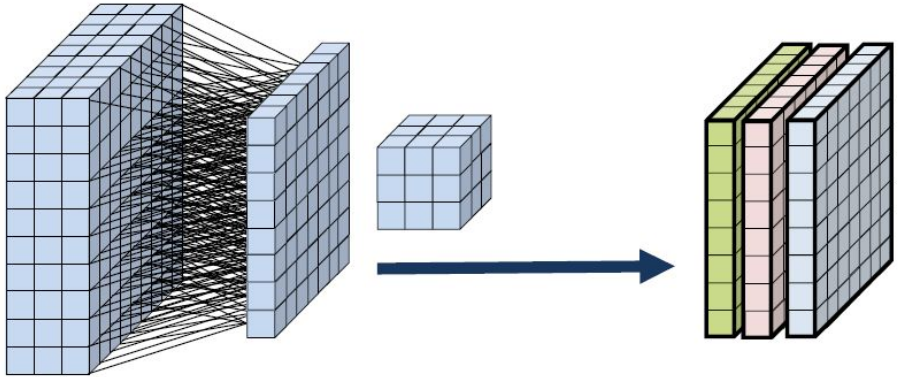
Consider a second filter,



Convolutional Layer



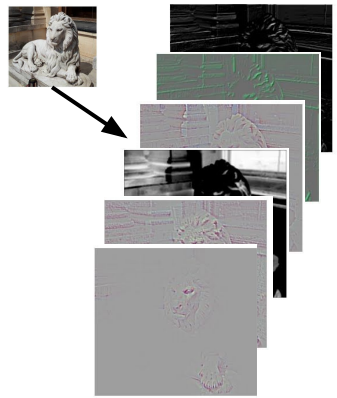
… and a third filter,



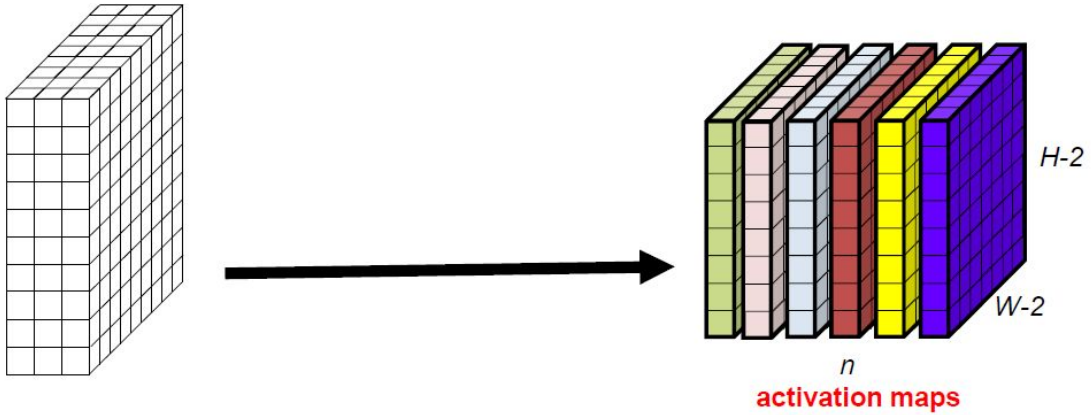
The *n* activation maps are stacked forming a (W-2) x (H-2) x n “new image”!

Adapted from Raul Feitosa (PUC-Rio)

Convolutional Layer



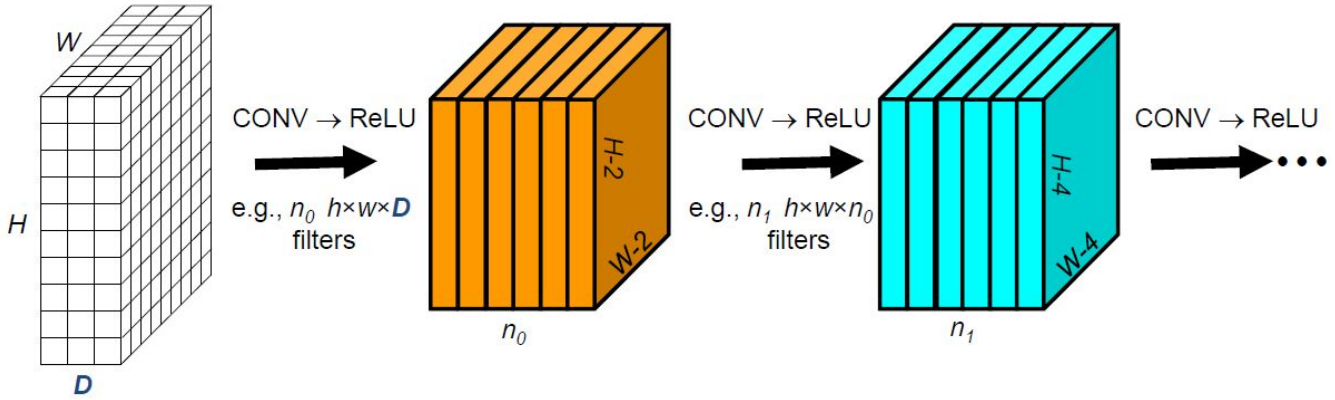
… up to *n*-th filters:



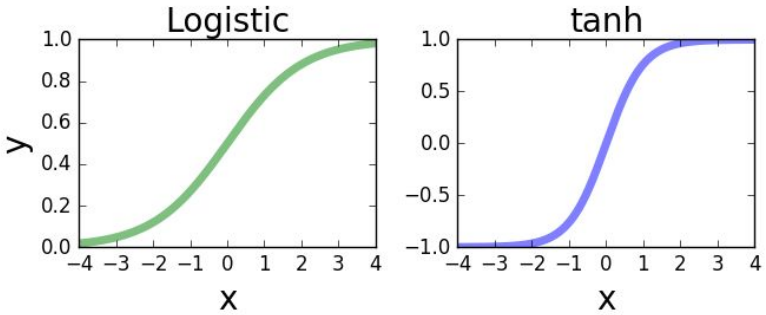
The *n* activation maps are stacked forming a (W-2) × (H-2) × n “new image”!

Basic CNN

A ConvNet is a sequence of Convolutional Layers, interspersed with **activation functions**.



Activation Functions for CNNs

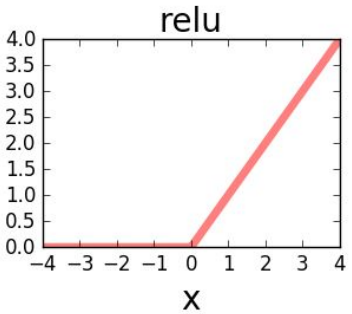


- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron.
- They “kill” the gradients when saturated

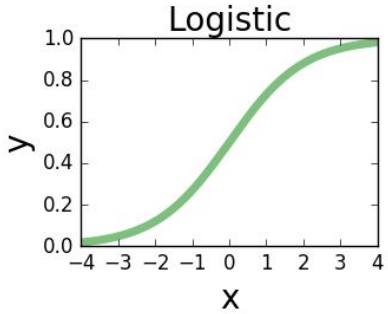
Recommended reading (Dr Jason Brownlee):

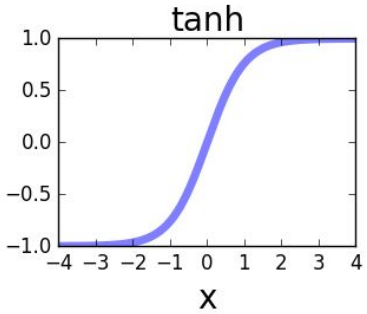
- https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/
- https://machinelearningmastery.com/how-to-fix-vanishing-gradients-using-the-rectified-linear-activation-function/

Activation Functions for CNNs → Rectified Linear Unit (ReLU)



$$f(x) = max(0, x)$$





- Does not saturate in the positive region.
- Very computationally efficient
- Converges much faster than sigmoid/tanh (≅ 6×)

Shortcoming:

1) not zero-centered output

Recommended reading (Dr Jason Brownlee):

- https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/
- https://machinelearningmastery.com/how-to-fix-vanishing-gradients-using-the-rectified-linear-activation-function/

Outline

1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - **Stride and Padding**
    - Pooling
    - Loss functions
3. Convolutional Neural Networks
    - Typical Architecture and Training Process
    - Data Augmentation
    - Transfer Learning and Fine Tuning

Stride

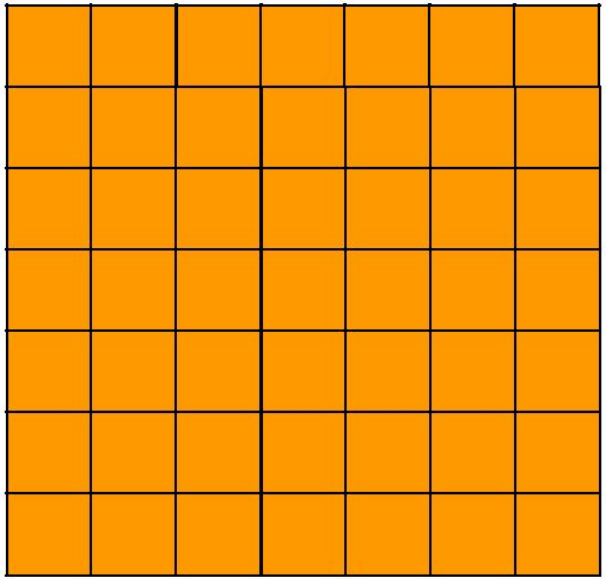
Refers to the number of pixels by which the filter (kernel) moves across the input image during the convolution operation.

- A stride of 1 means the filter moves one pixel at a time, resulting in overlapping receptive fields.
- A stride greater than 1 (e.g., stride of 2) causes the filter to skip pixels as it moves, resulting in a smaller output size.
- Using a larger stride can reduce the spatial dimensions of the output feature maps, leading to faster computations and potentially reducing overfitting by reducing spatial redundancy.

Convolutional Layers

Stride

Stride = 1

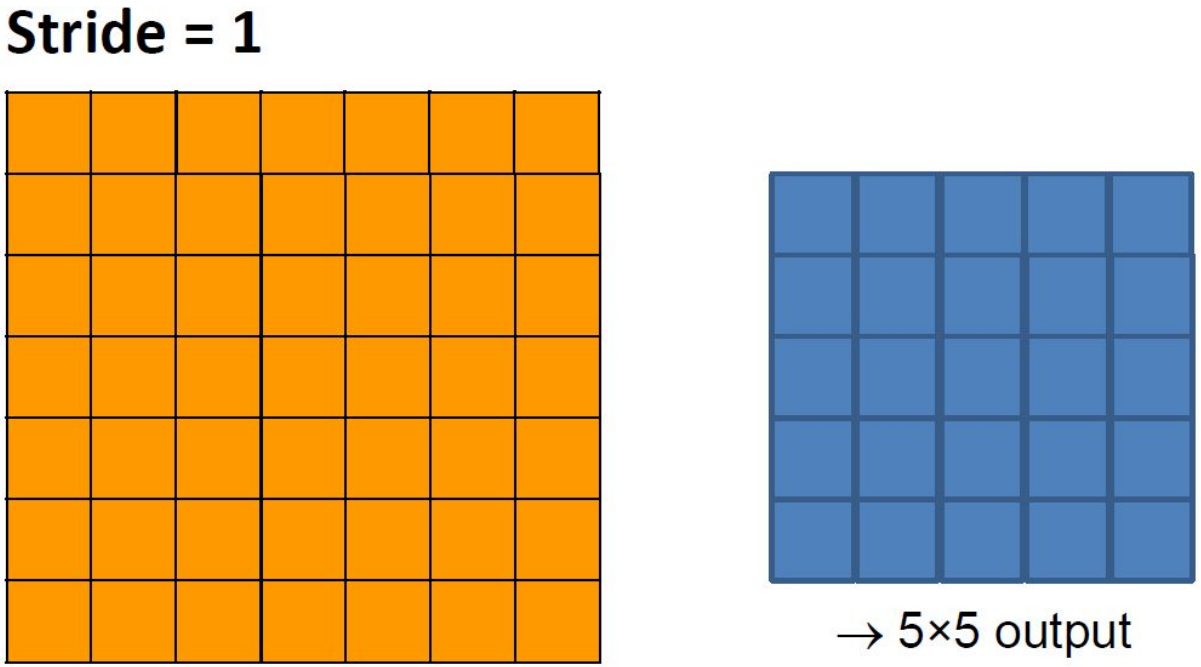


7x7 input (spatially) and 3×3 filter

What will the spatial dimension of the input image after the convolution?

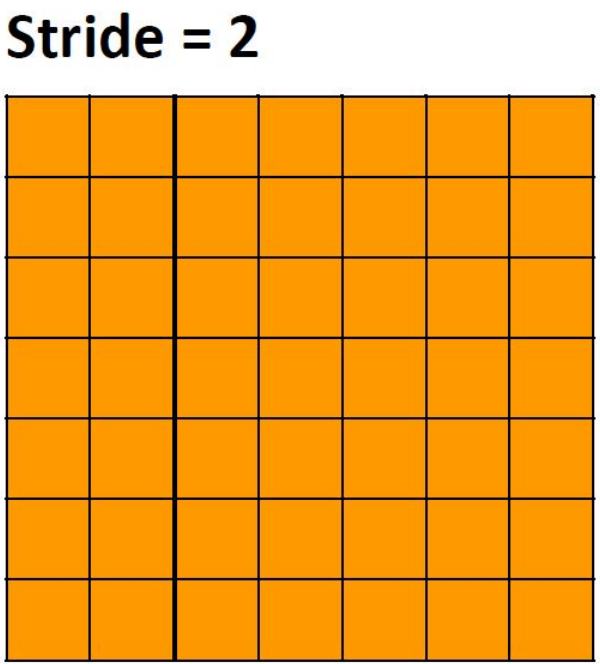
Convolutional Layers

Stride



Convolutional Layers

Stride



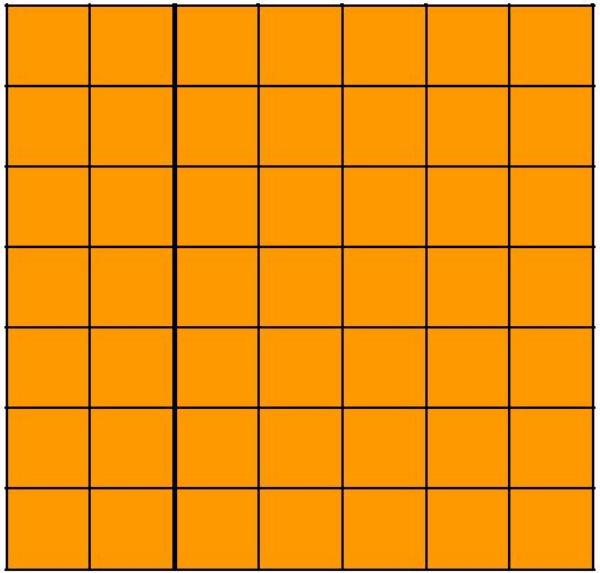
7x7 input (spatially) and 3×3 filter

And now?

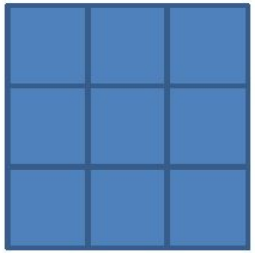
Convolutional Layers

Stride

Stride = 2



7x7 input (spatially) and 3×3 filter



→ 3×3 output

and for stride=3?

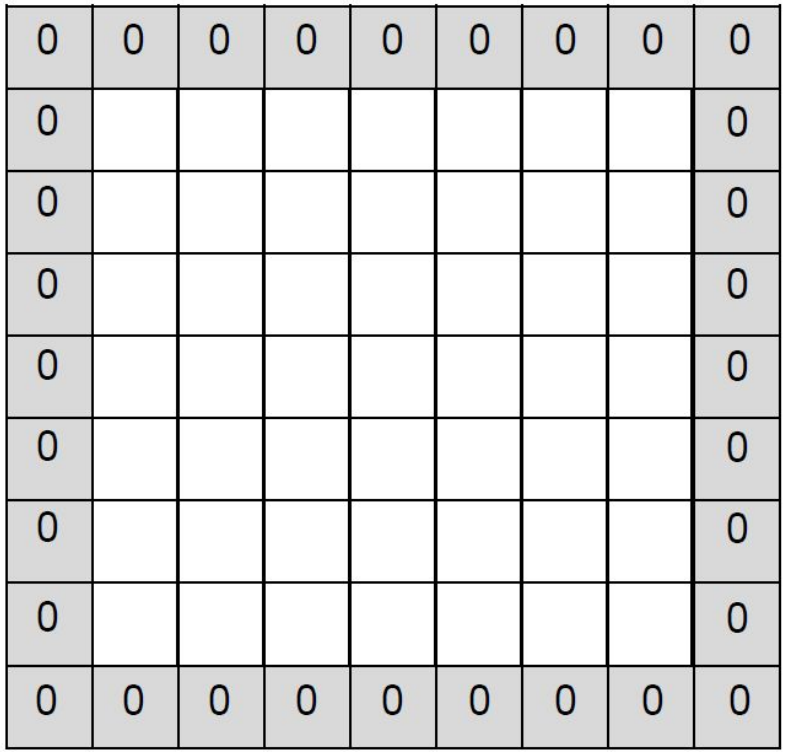
Padding

Is the process of adding extra pixels (usually zeros) around the input image before applying convolutional operations.

- The purpose of padding is to control the spatial dimensions of the output feature maps after convolution.
- With padding, the output feature maps can have the same spatial dimensions as the input image (if using "same" padding), or they can be smaller (if using "valid" padding).
- "Same" padding adds zeros around the input image so that the output size matches the input size when using a stride of 1.
- "Valid" padding, on the other hand, means no padding is added, resulting in output feature maps that are smaller than the input size, especially when combined with a stride greater than 1.

Convolutional Layers

Padding

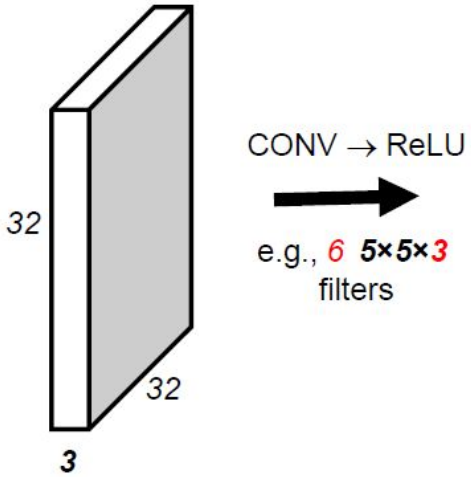


e.g. input 7×7 and a 3×3 filter, s=1
pad with 1 pixel border
→ what is the output?
7×7 output!

In general, CONV layers with s=1, zero-padding with (h-1)/2 (to preserve size spatially)
e.g. h = 3 → zero pad with 1
h = 5 → zero pad with 2
h = 7 → zero pad with 3

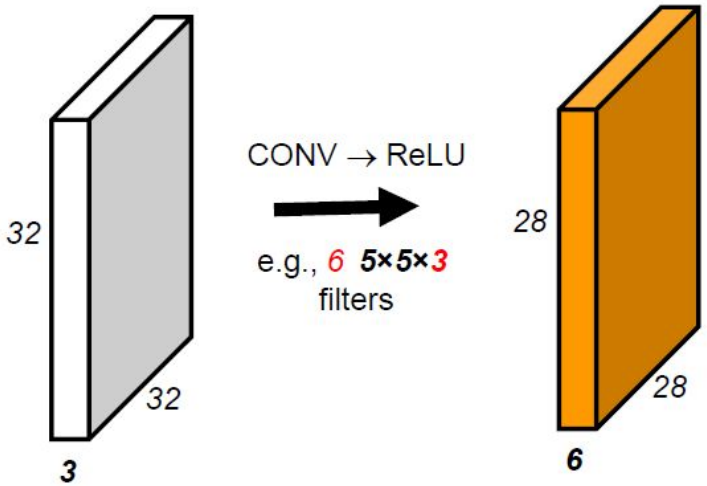
Spatial Dimension Along the Layers

E.g., a *32×32×3 input* convolved with *5×5* filters with no padding shrinks the volume spatially.



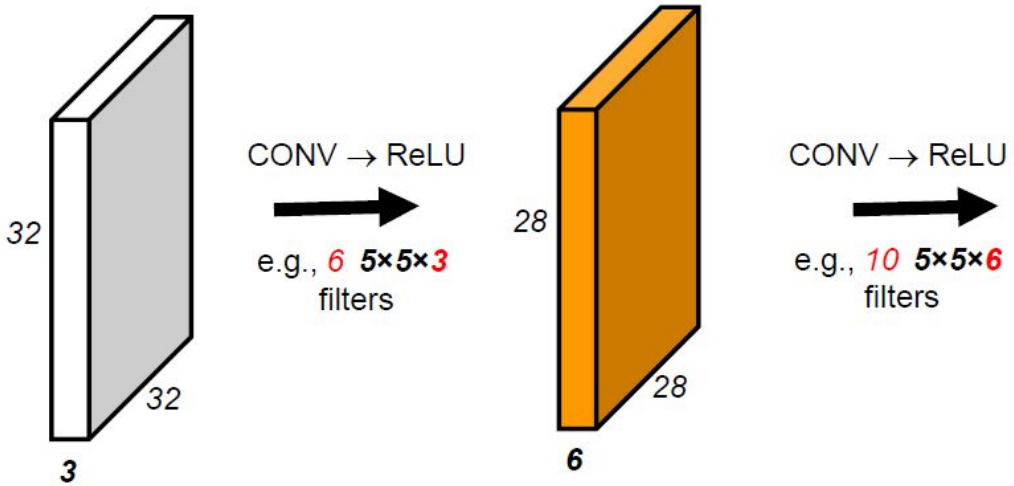
Spatial Dimension Along the Layers

E.g., a *32×32×3 input* convolved with *5×5* filters with no padding shrinks the volume spatially.



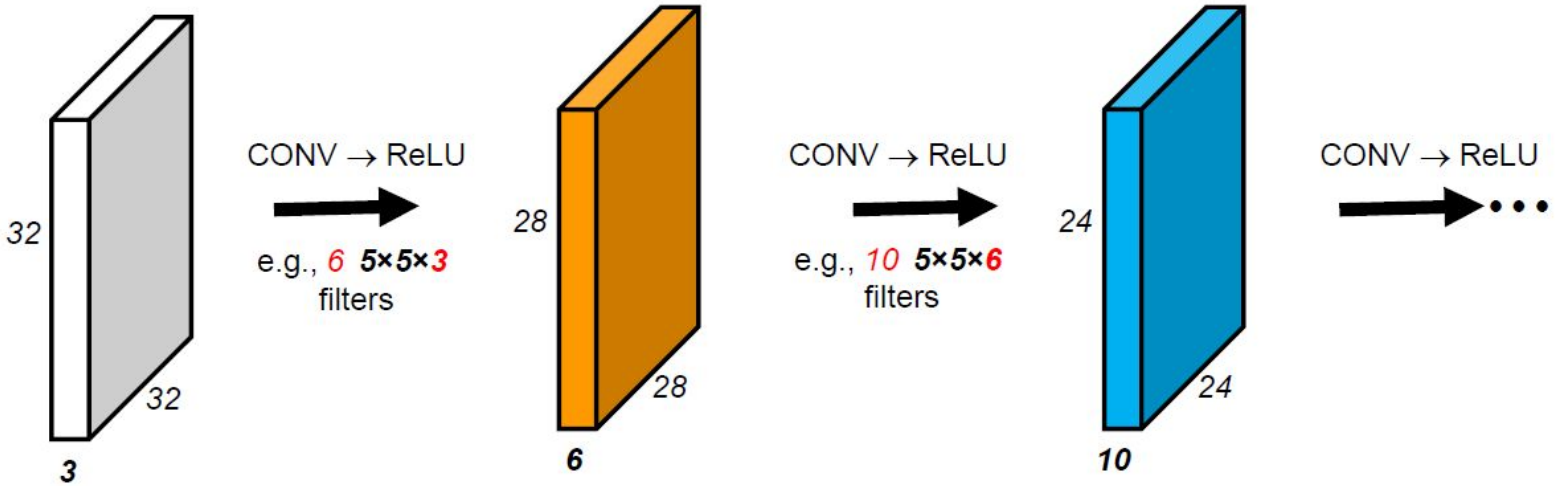
Spatial Dimension Along the Layers

E.g., a *32×32×3 input* convolved with *5×5* filters with no padding shrinks the volume spatially.



Spatial Dimension Along the Layers

E.g., a *32×32×3 input* convolved with *5×5* filters with no padding shrinks the volume spatially.



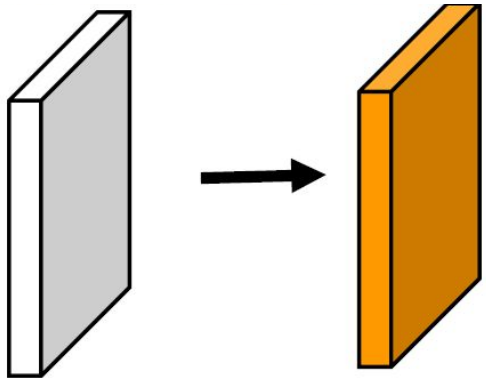
Spatial Dimension Along the Layers

E.g., a *32×32×3 input*

10 5×5 filters

stride= 1,

pad 2



Output volume?

Number of parameters?

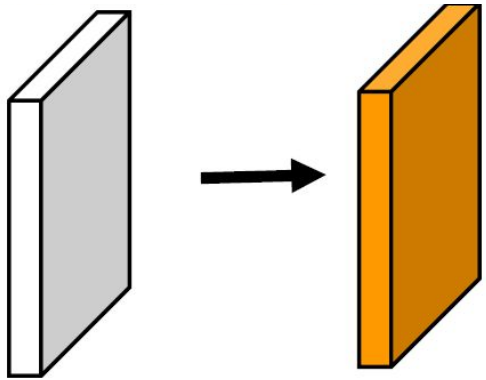
Spatial Dimension Along the Layers

E.g., a *32×32×3 input*

10 5×5 filters

stride= 1,

pad 2



Output volume?

→ *10240*

32*32*10

Number of parameters?

→ *10×(5×5×3+1)= 760*

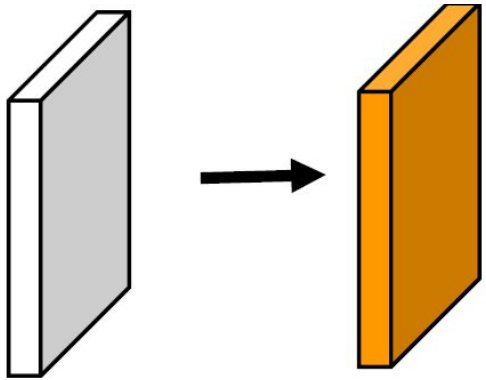
Spatial Dimension Along the Layers

E.g., a *32×32×3 input*

10 5×5 filters

stride= 1,

pad 2



Output volume?

→ *10240*

32*32*10

Number of parameters?

→ *10×(5×5×3+1)= 760*

Number of parameters for a fully connected layer?

→ *(32×32×3+1)×(32×32×3)×10≅10^8*

Adapted from Raul Feitosa (PUC-Rio)

Spatial Dimension Along the Layers - Summary

- Input size: $H_1 \times W_1 \times D_1$
- Hyperparameters:
  - number of filters k,
  - filter size h,
  - stride s,
  - *amount of zero padding* p
- *Produces a volume of size* $H_2 \times W_2 \times D_2$
  - $W_2=(W_1-h+2p)/s+1$
  - $H_2=(H_1-h+2p)/s+1$
  - $D_2=k$
- Each filter introduces h^2D_1 weights per filter, for a total of $h^2D_1\,k$ weights and k biases.
- The d-th slice of the output (of size $H_2 \times W_2$) is the result of performing a convolution of the d-th filter over the input volume with stride s, and then offset by the d-th bias.

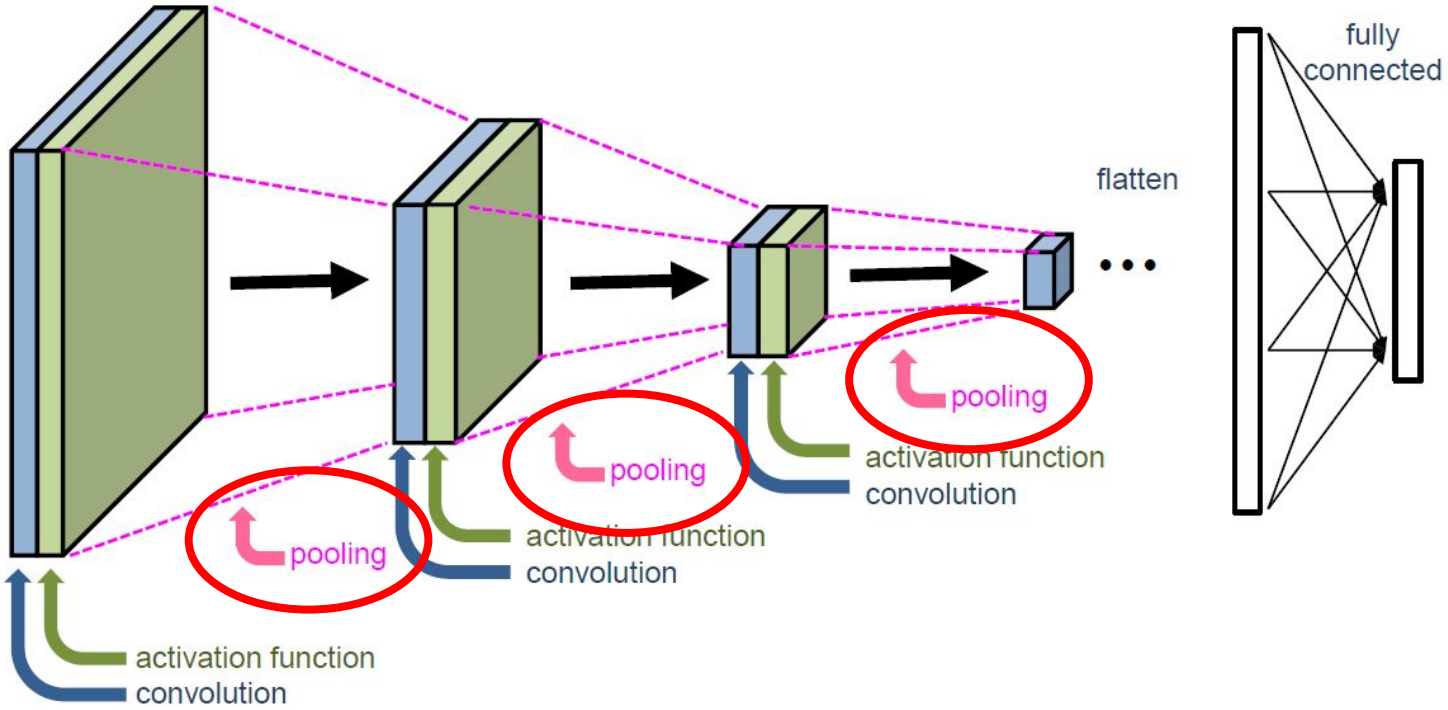
Adapted from Raul Feitosa (PUC-Rio)

Outline

1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - Stride and Padding
    - **Pooling**
    - Loss functions
3. Convolutional Neural Networks
    - Typical Architecture and Training Process
    - Data Augmentation
    - Transfer Learning and Fine Tuning

Convolutional Neural Networks

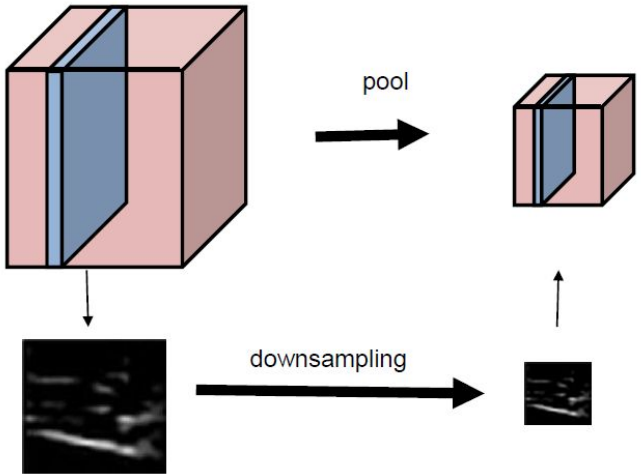
Typical Architecture



Adapted from Raul Feitosa (PUC-Rio)

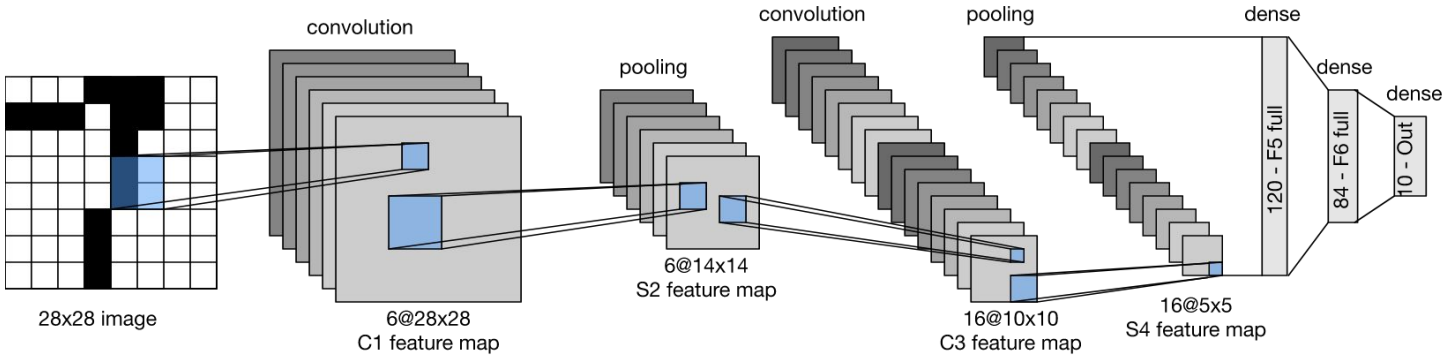
Pooling

- Replaces the output of the net at a certain location with a summary **statistic** of the nearby outputs.
- Helps to make the representation become approximately invariant to small translations of the input



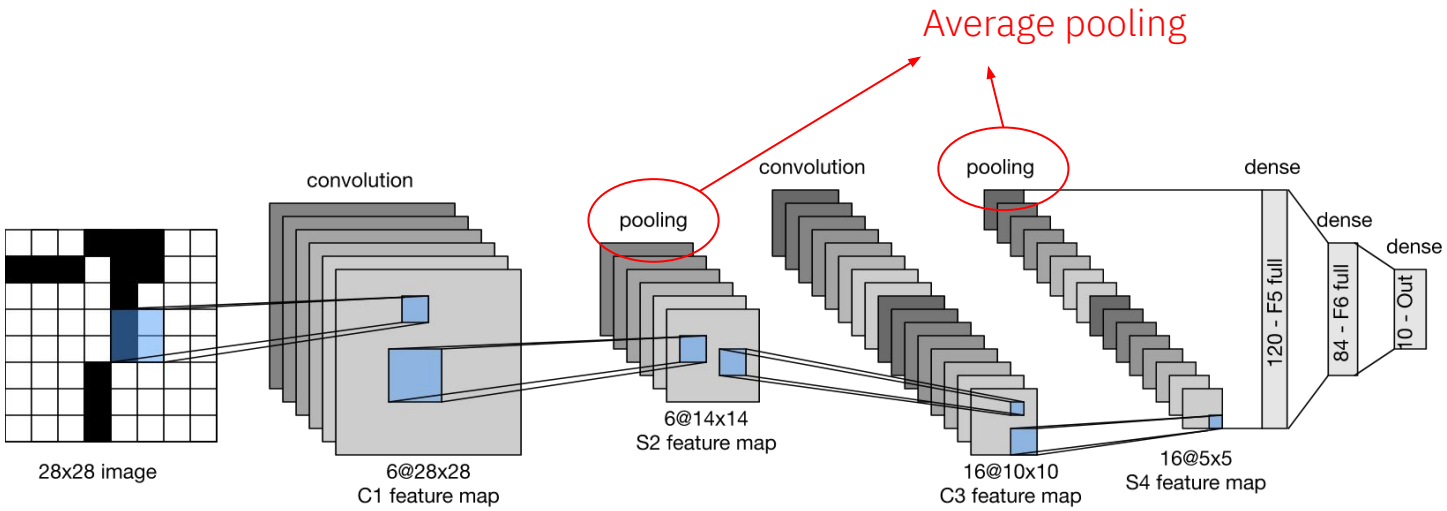
Dimensionality Reduction

Average Pooling (Le Net)



Source: https://d2l.ai/chapter_convolutional-neural-networks/lenet.html

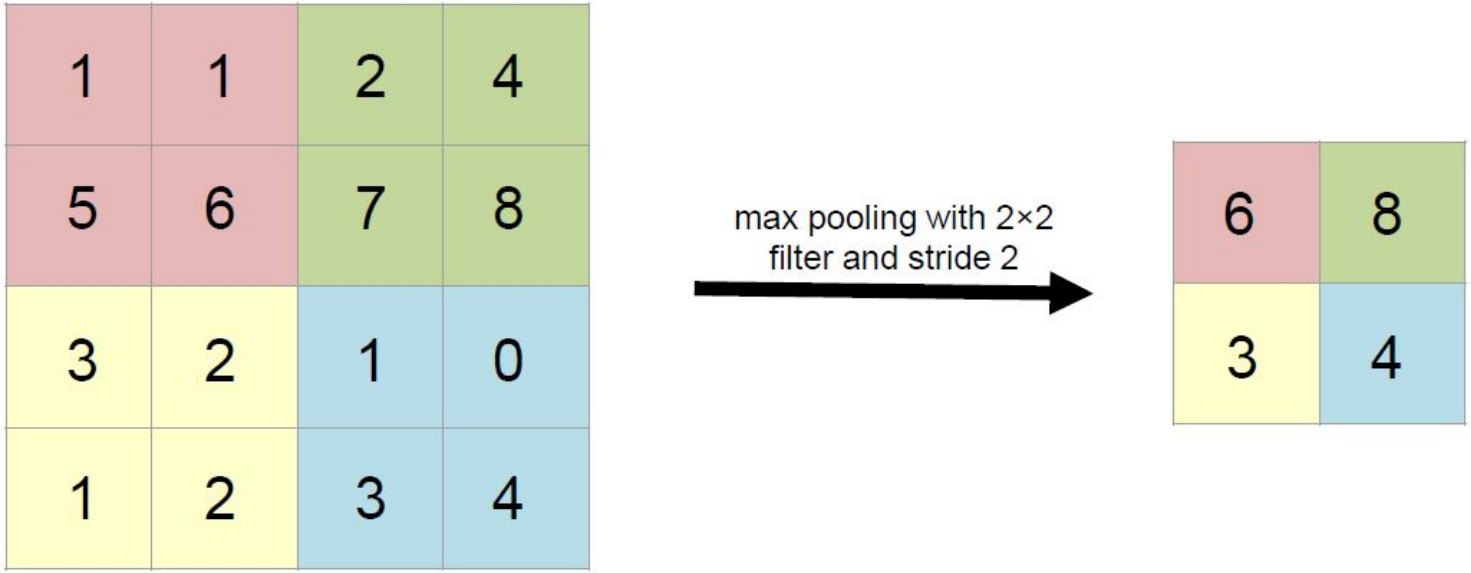
Average Pooling (Le Net)



What happen when we apply low-pass filters?

Source: https://d2l.ai/chapter_convolutional-neural-networks/lenet.html

Modern Architectures - Max Pooling



Outline

1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - Stride and Padding
    - Pooling
    - **Loss function**
    - Training strategies
3. Convolutional Neural Networks
    - Typical Architecture
    - Data Augmentation
    - Transfer Learning and Fine Tuning

Cross-Entropy Loss

Recall the loss function concept:

$$L(\boldsymbol{W}) = \frac{1}{N}\sum_i \underbrace{L_i(\boldsymbol{W}, \boldsymbol{x}_i, y_i)}_{\textbf{Data loss}} + \underbrace{\lambda\, R(\boldsymbol{W})}_{\textbf{Regularization}}$$

Data loss
match between
model and data

Regularization
model complexity

Cross-Entropy Loss

Recall the loss function concept:

$$L(\boldsymbol{W}) = \frac{1}{N}\sum_i \underbrace{L_i(\boldsymbol{W}, \boldsymbol{x}_i, y_i)}_{\text{Data loss}} + \underbrace{\lambda\, R(\boldsymbol{W})}_{\text{Regularization}}$$

Data loss
match between model and data

Regularization
model complexity

The cross-entropy loss is given by

$$L_i(\boldsymbol{W}, \boldsymbol{x}_i, y_i) = -\log p(y_i|\boldsymbol{W}, \boldsymbol{x}_i)$$

I.e., the log probability assigned by the model to the true class.

For softmax

$$L_i(\boldsymbol{W}, \boldsymbol{x}_i, y_i) = -\log \frac{e^{\boldsymbol{w}_{y_i}^T \boldsymbol{x}}}{\sum_k e^{\boldsymbol{w}_k^T \boldsymbol{x}}}$$

Cross-Entropy Loss

Example:

If the probabilities assigned by the classifier to the training sample x_i were:

 = .7  =.2  =.1

and the true label y_i is  ,

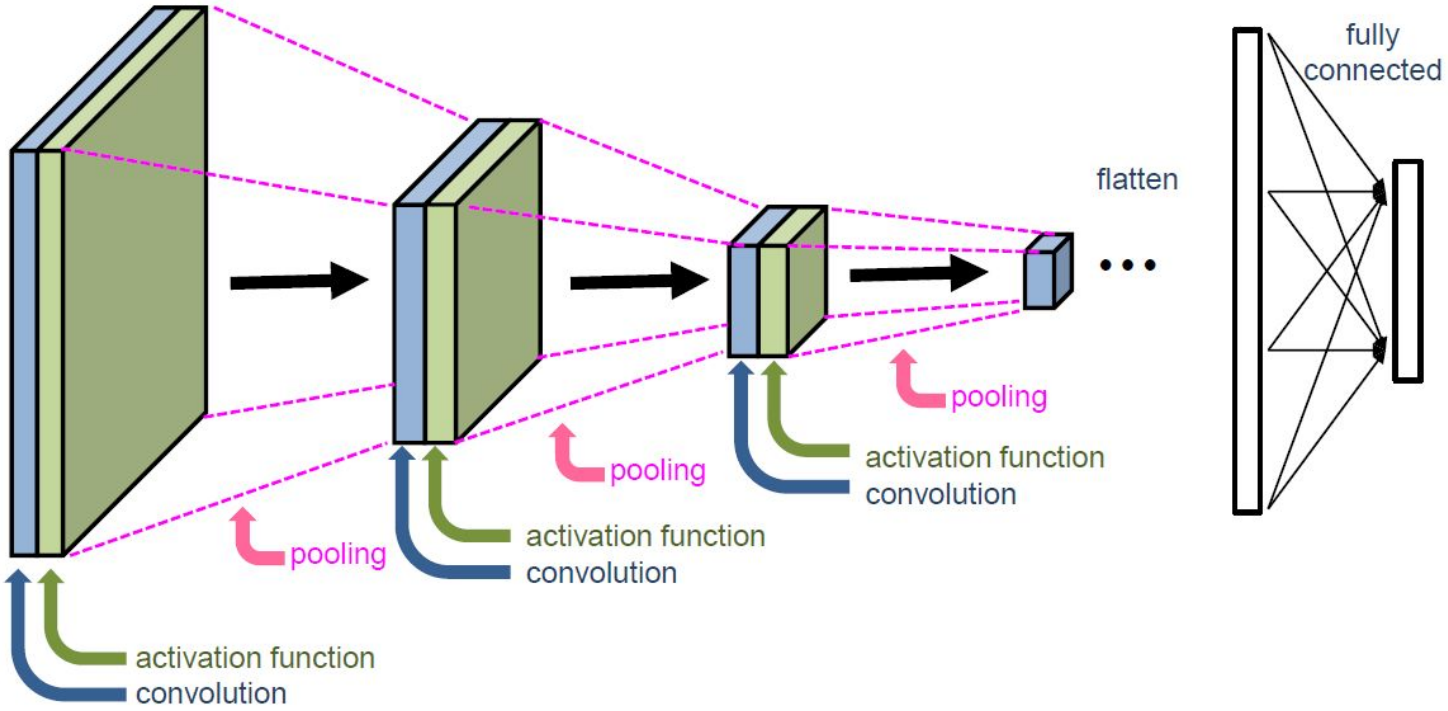
then the cross entropy of sample (x_i, y_i) is

$$L_i = -\log(.7) = 0.15$$

Outline

1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - Stride and Padding
    - Pooling
    - Loss functions
3. **Convolutional Neural Networks**
    - Typical Architecture and Training Process
    - Data Augmentation
    - Transfer Learning and Fine Tuning

Typical Architecture



Training a CNN from scratch

1. Gather an image labeled dataset
2. Preprocess the data (resizing, normalization, split dataset)
3. Model Architecture Design
    - Number of convolutional layers, pooling layers, activation functions, and fully connected layers
    - Decide on the output layer (e.g. softmax for classification, sigmoid for binary classification).
4. Data Augmentation (optional, but recommended)
5. Model Training
    - Define an appropriate optimizer (such as Adam or SGD), a loss function (e.g. cross entropy), and optional metrics (e.g., accuracy).
    - Define the number of epochs and the batch size.
    - Train the CNN model using Feedforward/Backpropagation steps
    - Monitor training progress by observing metrics like loss and accuracy on training and validation sets
6. Hyperparameter Tuning
    - Fine-tune hyperparameters such as learning rate, batch size, and regularization techniques (e.g., dropout) to optimize model performance.

Data Augmentation

Improves generalization ability!

- Generates similar but distinct training examples
- Expands the size of the training set
- Motivated by the fact that random tweaks allow to less rely on certain attributes

Other important concepts

1. **Dropout** https://d2l.ai/chapter_multilayer-perceptrons/dropout.html
2. **Batch Normalization**
   https://d2l.ai/chapter_convolutional-modern/batch-norm.html
3. **Data Augmentation**

   https://d2l.ai/chapter_computer-vision/image-augmentation.html

4. **Transfer Learning & Fine Tuning**

   https://d2l.ai/chapter_computer-vision/fine-tuning.html

Outline

1. Introduction
2. Basic Concepts
    - Convolutional Perceptron
    - Convolutional Layer
    - Stride and Padding
    - Pooling
    - Loss functions
3. Convolutional Neural Networks
    - Typical Architecture and Training Process
    - Data Augmentation
    - **Transfer Learning and Fine Tuning**

Transfer Learning - Motivation

1. CNNs might contain hundreds of millions of parameters to learn
2. This imposes a huge demand on labeled samples
3. Labeled datasets are scarce
4. Training CNNs from scratch may be impractical

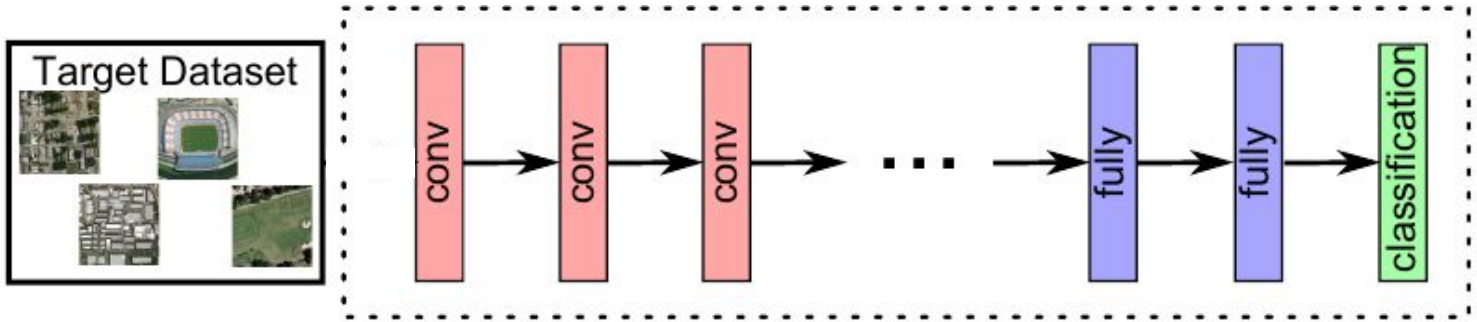
Use existing ConvNets as feature extractors!

Strategies to Exploit ConvNets

1 - Training from scratch

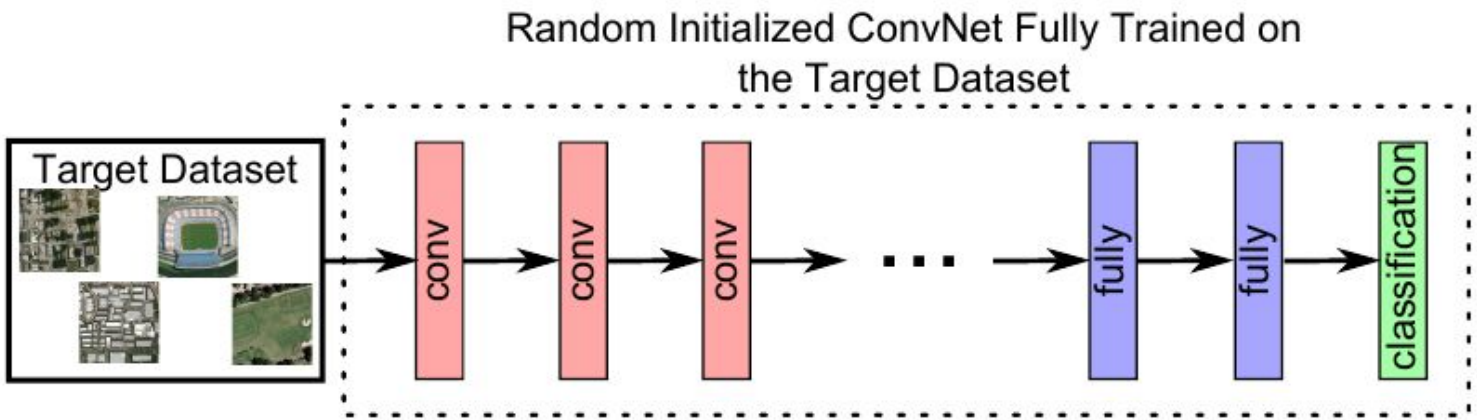
Strategies to Exploit ConvNets

1 - Training from scratch



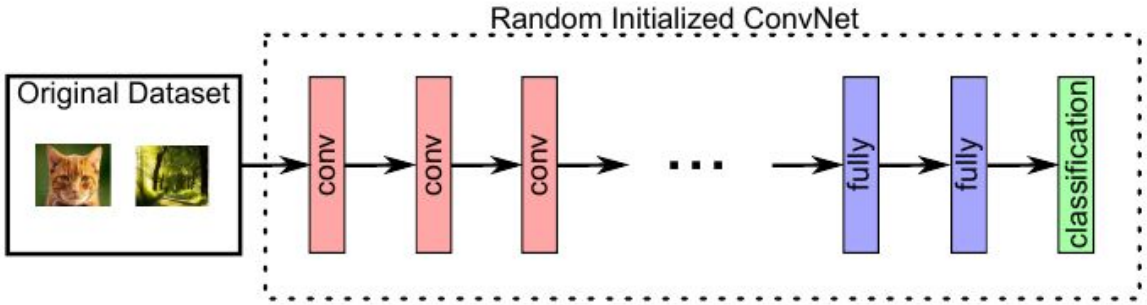
Strategies to Exploit ConvNets

1 - Training from scratch



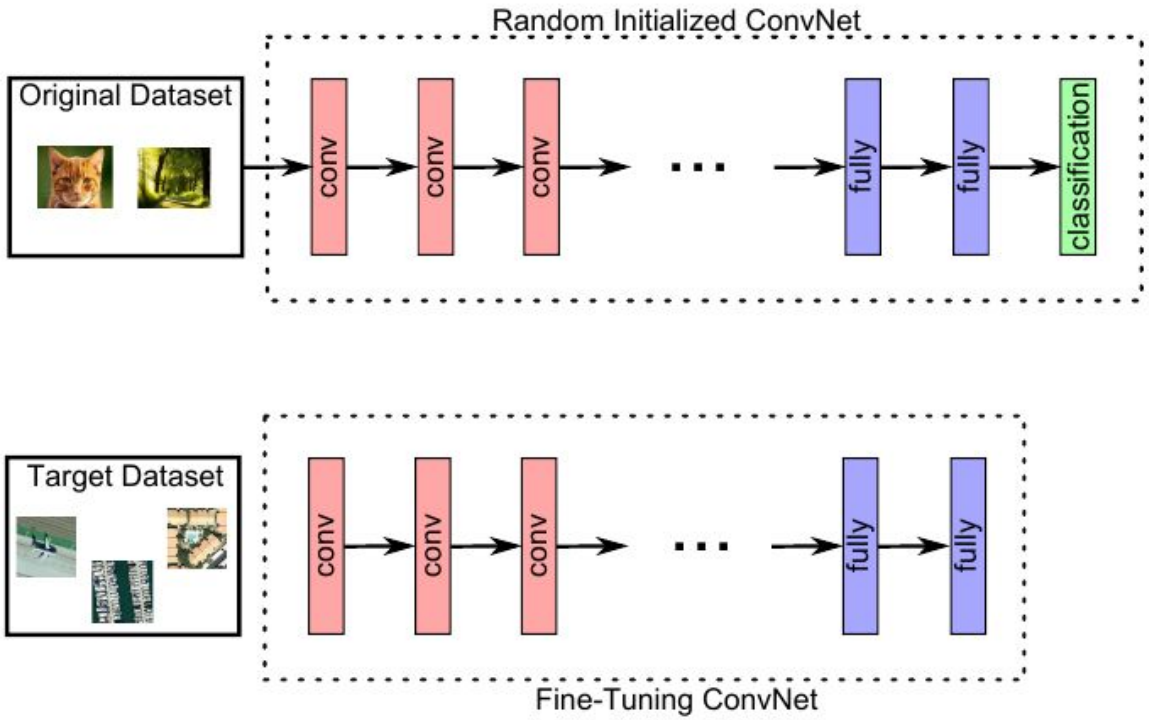
Strategies to Exploit ConvNets

2 - Fine-Tuning



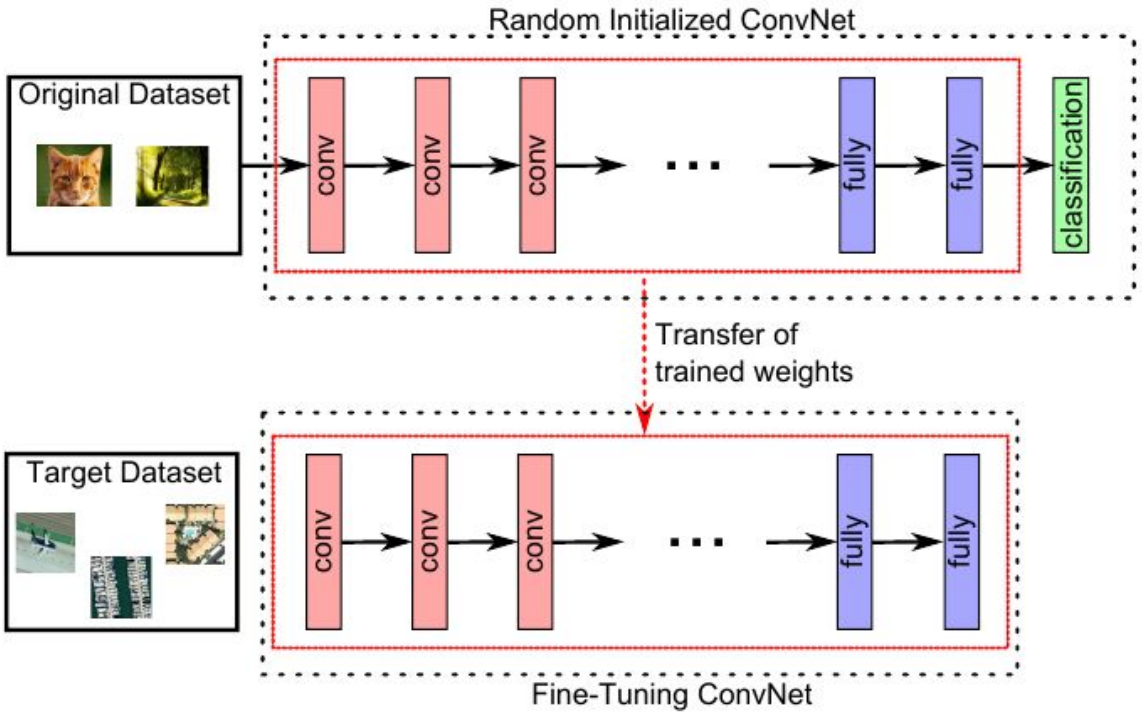
Strategies to Exploit ConvNets

2 - Fine-Tuning



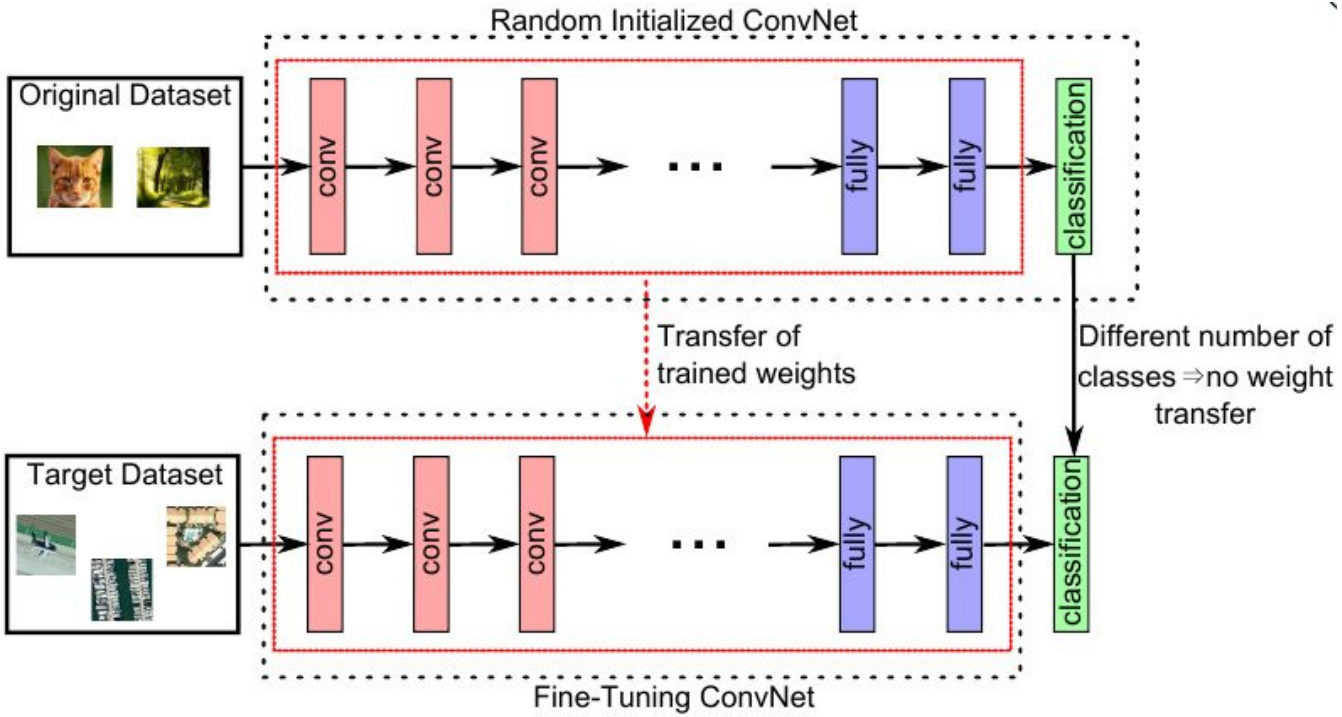
Strategies to Exploit ConvNets

2 - Fine-Tuning



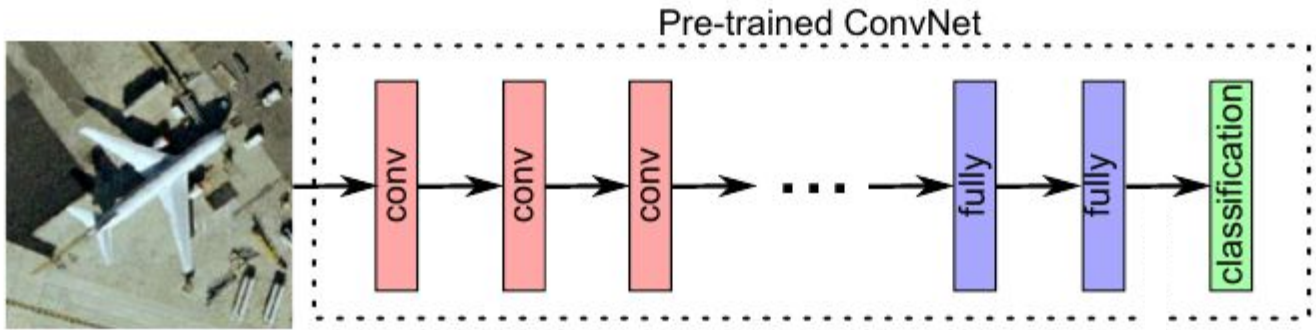
Strategies to Exploit ConvNets

2 - Fine-Tuning



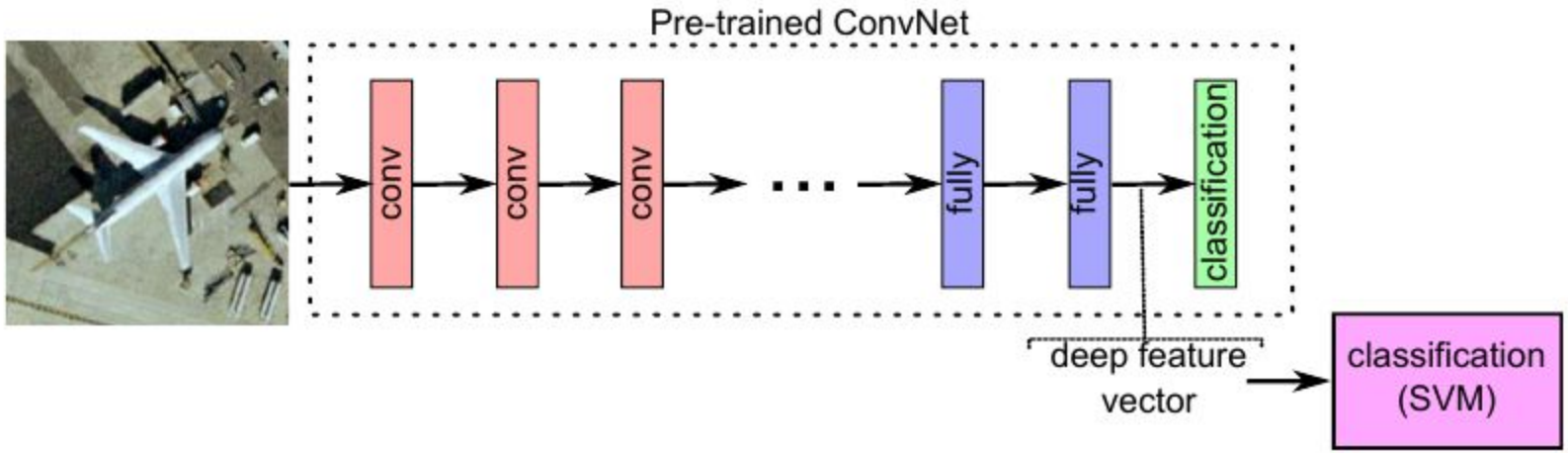
Strategies to Exploit ConvNets

3 - Feature Extraction

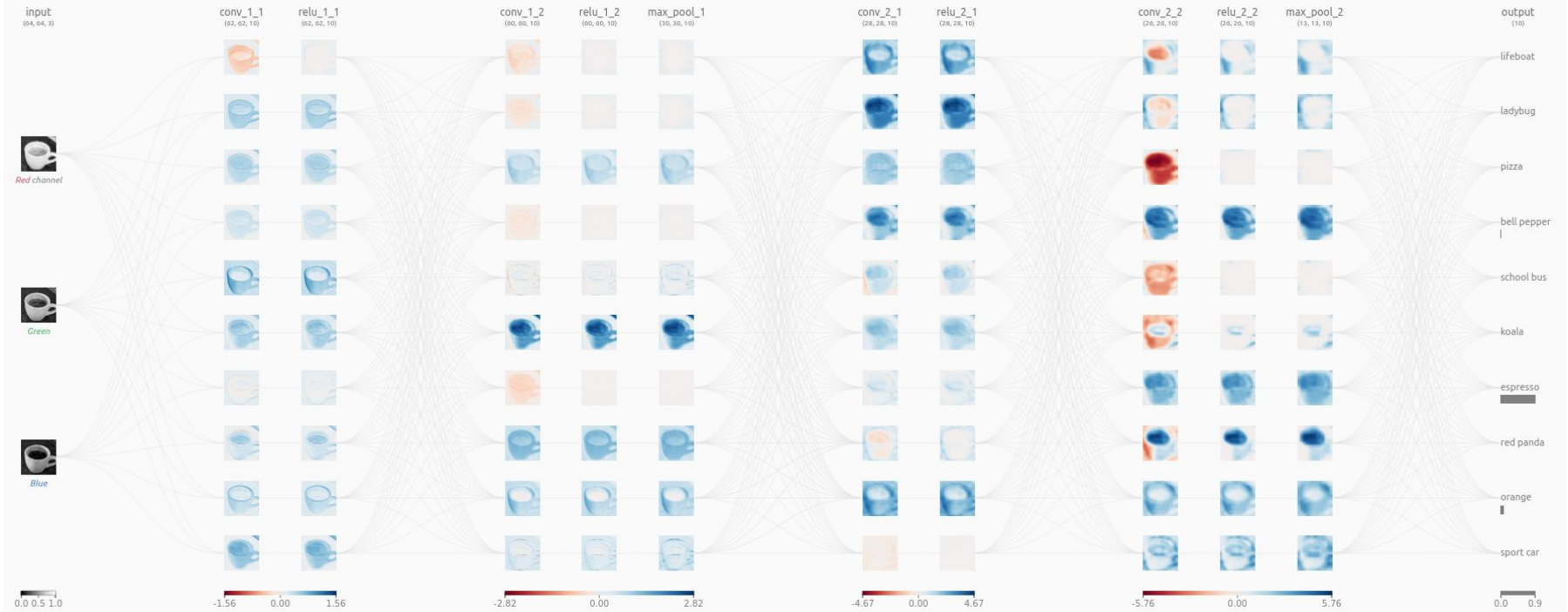


Strategies to Exploit ConvNets

3 - Feature Extraction



CNN Explainer



https://poloclub.github.io/cnn-explainer/

CNNs with Pytorch

1. Convolutional layers receive tensors with the follow dimensions:
    - BATCH_SIZE, INPUT_CHANNELS, WIDTH, HEIGHT
2. No need to specify the number of input channels in the Max-Pooling and ReLU layers
3. You need to linearize the tensors manually before the first Fully Connected layer using the view() function
4. You need to pass the number of channels from the previous Convolutional Layer to the 2D Batch Normalization layers.

Read function documentation before implementing

CNNs with Pytorch

- Convolucional Layers (torch.nn.Conv2d)
  - https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html#torch.nn.Conv2d
- Pooling Layers (torch.nn.MaxPool2d)
  - https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html#torch.nn.MaxPool2d
- Camadas de Batch Normalization (torch.nn.BatchNorm2d)
  - https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html#torch.nn.BatchNorm2d
- ReLU Activation Function (torch.nn.ReLU)
  - https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html#torch.nn.ReLU
- Fully Connected Layers (torch.nn.Linear)
  - https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear

Summary

1. Overview
- Specialized deep learning models for processing structured grid data, primarily used in computer vision tasks.

2. Key Components
- Convolutional Layers: Apply filters to extract features.
- Pooling Layers: Downsample and reduce spatial dimensions.
- Activation Functions: Introduce non-linearity (e.g., ReLU).
- Fully Connected Layers: Perform classification/regression.

3. Architecture
- Input Layer: Raw data (e.g., images, sequences).
- Convolutional Layers: Feature extraction.
- Pooling Layers: Spatial downsampling.
- Fully Connected Layers: Decision-making.

4. Training
- Backpropagation: Update weights using gradient descent.
- Loss Functions: Measure model performance (e.g., cross-entropy).
- Optimization: Techniques like Adam, SGD for model convergence.

Summary

5. Applications

- Image Classification: Identify objects in images.
- Object Detection: Locate and classify objects within images.
- Image Segmentation: Assign labels to each pixel in an image.

6. Benefits

- Hierarchical feature learning.
- Parameter sharing reduces overfitting.
- Effective in handling spatial relationships.

7. Considerations

- Model size and complexity.
- Data augmentation for better generalization.
- Hyperparameter tuning for optimal performance.